

Advanced Python Programming

Comprehensive Guide with Book Definitions & Examples

Python Advanced Concepts Reference

May 8, 2026

Contents

1	Python Basics - Variables, Strings & Data Types	4
1.1	Variables in Python	4
1.1.1	Variable Rules and Conventions	4
1.1.2	Naming Conventions	4
1.2	Data Types in Python	5
1.3	Strings in Python	6
1.3.1	String Creation and Methods	6
1.3.2	String Formatting Methods Comparison	7
1.4	None and Boolean Values	7
2	Lists, Tuples, Dictionaries & Sets	9
2.1	Lists	9
2.1.1	List Creation and Basic Operations	9
2.1.2	List Methods and Operations	10
2.1.3	List Comprehensions	10
2.1.4	Advanced List Operations	11
2.2	Tuples	12
2.3	Dictionaries	13
2.3.1	Advanced Dictionary Types	14
2.4	Sets	14
3	Arrays and Matrices in Python	16
3.1	Python's array Module	16
3.2	Lists as Matrices (Nested Lists)	17
4	NumPy - Numerical Python	19
4.1	NumPy Arrays	19
4.2	NumPy Array Operations	20
4.3	Indexing and Slicing	20
4.4	Reshaping and Stacking	21
4.5	Linear Algebra with NumPy	22
5	Pandas - Data Analysis Library	23
5.1	Pandas Series	23
5.2	Pandas DataFrame	24
5.3	DataFrame Operations	24
5.4	Data Manipulation	25
5.5	Grouping and Aggregation	26
5.6	Handling Missing Data	27

5.7	Merging and Joining DataFrames	27
5.8	Time Series with Pandas	28
5.9	Input/Output Operations	29
5.10	Pandas Cheat Sheet	30
5.11	NumPy vs Pandas Comparison	30
6	Operator Overloading in Python	31
6.1	Simple Explanation	31
6.2	Common Operator Overloading Methods	31
6.3	Examples	31
6.3.1	Example 1: Vector Class with Operator Overloading	31
6.4	Key Points to Remember	34
7	Method Overloading in Python	35
7.1	Simple Explanation	35
7.2	Techniques for Python Method Overloading	35
7.3	Example: Using @singledispatch for Type-Based Overloading	35
7.4	Example: Real-World Employee Salary Calculator	37
8	Abstract Base Class (ABC) in Python	38
8.1	Simple Explanation	38
8.2	Key ABC Components	38
8.3	Example: Payment Processing System	38
8.4	Benefits of Using ABCs	41
9	Iterators in Python	42
9.1	Simple Explanation	42
9.2	Iterator vs Iterable	42
9.3	Example: Custom Fibonacci Iterator	42
9.4	Example: Infinite Iterator with islice	44
9.5	Common Iterator Patterns	44
10	Generators in Python	45
10.1	Simple Explanation	45
10.2	Generator vs Regular Function vs Iterator Class	45
10.3	Example: Generator Pipeline for Data Processing	45
10.4	Example: Generator with yield from (Delegation)	47
10.5	Example: Generators for Large File Processing	47
10.6	Generator Methods	48
11	Error Handling in Python	50
11.1	Simple Explanation	50
11.2	Python Exception Hierarchy	50
11.3	Example: Complete Error Handling with Custom Exceptions	51
11.4	The try-except-else-finally Structure	53
11.5	Exception Handling Best Practices	53

12 Context Managers in Python	55
12.1 Simple Explanation	55
12.2 Example: Custom Context Manager	55
13 Decorators in Python	57
13.1 Simple Explanation	57
13.2 Example: Practical Decorators	57
14 Descriptors and Properties	60
14.1 Example: Property Decorator and Custom Descriptor	60
15 Metaclasses in Python	63
15.1 Simple Explanation	63
15.2 Example: Using Metaclasses	63
16 Asynchronous Programming with asyncio	66
16.1 Example: Async/Await with asyncio	66
17 Python Advanced Concepts Cheat Sheet	69
17.1 Operator Overloading Quick Reference	69
17.2 Iterator vs Generator vs Generator Expression	69
17.3 Exception Handling Keywords	70
17.4 Common Python Magic Methods	70
17.5 Important Built-in Decorators	70
17.6 ABC (Abstract Base Classes) Quick Reference	70

Chapter 1

Python Basics - Variables, Strings & Data Types

Book Definitions

Source: "Python Crash Course" by Eric Matthes

"Variables are labels that you can assign to values. In Python, variables are created when you first assign a value to them. Python is dynamically typed, meaning you don't need to declare the type of a variable before using it."

Source: "Learning Python" by Mark Lutz

"Strings in Python are sequences of characters used to store and manipulate text. Python strings are immutable sequences, meaning once created, they cannot be changed in place."

1.1 Variables in Python

1.1.1 Variable Rules and Conventions

- **Must start** with a letter (a-z, A-Z) or underscore (`_`)
- **Cannot start** with a number
- **Can contain** letters, numbers, and underscores
- **Case-sensitive:** `name`, `Name`, and `NAME` are different
- **No reserved words:** Cannot use Python keywords (`if`, `for`, `class`, etc.)

1.1.2 Naming Conventions

Style	Example	Used For
snake_case	<code>user_name</code> , <code>total_count</code>	Variables, functions, methods
CamelCase	<code>MyClass</code> , <code>BankAccount</code>	Classes
UPPER_CASE	<code>MAX_SIZE</code> , <code>PI</code>	Constants
<code>_leading_underscore</code>	<code>_internal</code> , <code>_private_method</code>	Internal/private use

```

1 # Basic variable assignment
2 name = "Alice Johnson"
3 age = 25
4 height = 5.6
5 is_student = True
6
7 # Multiple assignment
8 x, y, z = 10, 20, 30
9 a = b = c = 100 # Same value to multiple variables
10
11 # Dynamic typing - variable can change type
12 value = 42
13 print(f"Type: {type(value).__name__}, Value: {value}")
14
15 value = "Now I'm a string"
16 print(f"Type: {type(value).__name__}, Value: {value}")
17
18 value = [1, 2, 3]
19 print(f"Type: {type(value).__name__}, Value: {value}")
20
21 # Swapping variables (Pythonic way)
22 a, b = 5, 10
23 a, b = b, a
24 print(f"After swap: a={a}, b={b}")
25
26 # Type checking
27 print(f"Is age integer? {isinstance(age, int)}")
28 print(f"Is name string? {isinstance(name, str)}")
29
30 # Variable scope
31 global_var = "I'm global"
32
33 def demonstrate_scope():
34     local_var = "I'm local"
35     print(f"Inside function: {global_var}") # Can access global
36     print(f"Inside function: {local_var}")  # Local variable
37
38 demonstrate_scope()
39 # print(local_var) # Error! Not accessible outside function

```

Listing 1.1: Variable Assignment and Usage

1.2 Data Types in Python

Type	Class	Description	Example
Integer	int	Whole numbers (unlimited size)	42, -17, 0
Float	float	Decimal numbers	3.14, -0.5, 1.0
Complex	complex	Complex numbers	3+4j
Boolean	bool	True or False	True, False
String	str	Text/characters	"Hello"
None	NoneType	Absence of value	None

```

1 # Integer operations
2 num1, num2 = 17, 5
3 print(f"Addition: {num1 + num2}")      # 22
4 print(f"Subtraction: {num1 - num2}")   # 12
5 print(f"Multiplication: {num1 * num2}") # 85
6 print(f"Division: {num1 / num2}")      # 3.4 (always float)
7 print(f"Floor Division: {num1 // num2}") # 3
8 print(f"Modulus: {num1 % num2}")       # 2
9 print(f"Power: {num1 ** num2}")        # 1419857
10
11 # Type conversion (casting)
12 print(f"Int to float: {float(42)}")    # 42.0
13 print(f"Float to int: {int(3.99)}")    # 3 (truncates)
14 print(f"String to int: {int('123')}")  # 123
15 print(f"String to float: {float('3.14')}") # 3.14
16 print(f"Number to string: {str(42)}")  # "42"
17 print(f"To boolean: {bool(1)}, {bool(0)}") # True, False
18
19 # Checking types
20 x = 42
21 print(f"x is int: {isinstance(x, int)}") # True
22 print(f"x is float: {isinstance(x, float)}") # False
23 print(f"x is number: {isinstance(x, (int, float, complex))}") # True

```

Listing 1.2: Working with Basic Data Types

1.3 Strings in Python

1.3.1 String Creation and Methods

```

1 # String creation
2 s1 = 'Single quotes'
3 s2 = "Double quotes"
4 s3 = '''Triple quotes for
5 multi-line strings'''
6 s4 = """Also triple quotes
7 for multi-line strings"""
8
9 # String concatenation and repetition
10 first = "Hello"
11 last = "World"
12 full = first + " " + last
13 print(f"Concatenation: {full}")        # Hello World
14 print(f"Repetition: {'Ha' * 3}")       # HaHaHa
15
16 # String indexing and slicing
17 text = "Python Programming"
18 print(f"First char: {text[0]}")        # P
19 print(f"Last char: {text[-1]}")        # g
20 print(f"First 6 chars: {text[:6]}")    # Python
21 print(f"Chars 7-11: {text[7:12]}")     # Progr
22 print(f"Every 2nd char: {text[::2]}")   # Pto rgamn
23 print(f"Reverse: {text[::-1]}")        # gnimmargorP nohtyP
24
25 # String methods
26 message = " Hello, Python World! "

```

```

27 print(f"Upper: {message.upper()}")           # HELLO, PYTHON WORLD!
28 print(f"Lower: {message.lower()}")           # hello, python world!
29 print(f"Title: {message.title()}")           # Hello, Python World!
30 print(f"Strip: '{message.strip()}'")         # 'Hello, Python World
    !',
31 print(f"Replace: {message.replace('Python', 'Java')}")
32 print(f"Split: {message.split(',')}")         # [' Hello', ' Python
    World! ']
33 print(f"Join: {'-'.join(['A', 'B', 'C'])}")   # A-B-C
34 print(f"Find: {message.find('Python')}")     # 9 (index)
35 print(f"Count: {message.count('o')}")        # 3
36 print(f"Starts with: {message.strip().startswith('Hello')}") # True
37 print(f"Ends with: {message.strip().endswith('!')}") # True
38 print(f"Is alpha: {'Hello'.isalpha()}")      # True
39 print(f"Is digit: {'123'.isdigit()}")        # True
40 print(f"Is alphanumeric: {'Hello123'.isalnum()}") # True
41
42 # f-strings (formatted strings) - Python 3.6+
43 name = "Alice"
44 age = 25
45 height = 5.6
46 print(f"My name is {name}, I'm {age} years old")
47 print(f"Pi is approximately {22/7:.4f}")     # 3.1429
48 print(f"Percentage: {0.856:.1%}")            # 85.6%
49 print(f"Hex: {255:x}, Binary: {255:b}")      # ff, 11111111
50 print(f"Padding: {name:<10}|{name:>10}|{name:^10}") # Left, right,
    center

```

Listing 1.3: String Operations and Methods

1.3.2 String Formatting Methods Comparison

Method	Example	Use Case
% formatting	"%s is %d" % ("Alice", 25)	Old style
.format()	"{} is {}".format("Alice", 25)	Python 2.6+
f-strings	f"{name} is {age}"	Python 3.6+ (recommended)

1.4 None and Boolean Values

```

1 # None - represents absence of value
2 result = None
3 print(f"None type: {type(None)}")           # <class 'NoneType'>
4
5 # Checking for None
6 def get_user(name):
7     if name == "admin":
8         return {"name": name, "role": "admin"}
9     return None

```

```

10
11 user = get_user("guest")
12 if user is None:
13     print("User not found")
14 elif user is not None:
15     print(f"Found user: {user}")
16
17 # Boolean values and operations
18 is_active = True
19 is_admin = False
20
21 print(f"AND: {is_active and is_admin}")    # False
22 print(f"OR: {is_active or is_admin}")      # True
23 print(f"NOT: {not is_active}")             # False
24
25 # Truthy and Falsy values
26 # Falsy: None, False, 0, 0.0, "", [], {}, set(), ()
27 # Truthy: Everything else
28
29 items = []
30 if not items:
31     print("List is empty (falsy)")
32
33 name = "Alice"
34 if name:
35     print("String is not empty (truthy)")
36
37 # Ternary operator (conditional expression)
38 age = 20
39 status = "Adult" if age >= 18 else "Minor"
40 print(f"Status: {status}")

```

Listing 1.4: Working with None and Booleans

Chapter 2

Lists, Tuples, Dictionaries & Sets

Book Definitions

Source: "Python Crash Course" by Eric Matthes

"Lists are ordered, mutable collections of items. Tuples are ordered, immutable collections. Dictionaries are unordered collections of key-value pairs. Sets are unordered collections of unique elements."

2.1 Lists

2.1.1 List Creation and Basic Operations

```
1 # Creating lists
2 empty_list = []
3 numbers = [1, 2, 3, 4, 5]
4 mixed = [1, "hello", 3.14, True]
5 nested = [[1, 2], [3, 4], [5, 6]]
6
7 # List from other sequences
8 chars = list("Python") # ['P', 'y', 't', 'h', 'o', 'n']
9 range_list = list(range(10)) # [0, 1, 2, ..., 9]
10
11 # Accessing elements
12 fruits = ["apple", "banana", "cherry", "date", "elderberry"]
13 print(f"First: {fruits[0]}") # apple
14 print(f>Last: {fruits[-1]}") # elderberry
15 print(f"Slice: {fruits[1:4]}") # ['banana', 'cherry', 'date']
16 print(f"Every 2nd: {fruits[::2]}") # ['apple', 'cherry', 'elderberry']
17
18 # Modifying lists
19 fruits[1] = "blueberry"
20 print(f"After modify: {fruits}")
21
22 fruits.append("fig") # Add to end
23 fruits.insert(2, "coconut") # Insert at position
24 fruits.extend(["grape", "honeydew"]) # Add multiple
25 print(f"After adds: {fruits}")
26
27 # Removing elements
28 fruits.remove("apple") # Remove by value
```

```

29 popped = fruits.pop()           # Remove and return last
30 popped_index = fruits.pop(2)    # Remove by index
31 del fruits[0]                   # Delete by index
32 fruits.clear()                  # Remove all
33
34 print(f"After clears: {fruits}")

```

Listing 2.1: List Operations

2.1.2 List Methods and Operations

Method	Description
append(x)	Add item x to end
extend(iterable)	Add all items from iterable
insert(i, x)	Insert x at position i
remove(x)	Remove first occurrence of x
pop(i=-1)	Remove and return item at index i
clear()	Remove all items
index(x)	Return index of first occurrence of x
count(x)	Count occurrences of x
sort(key=None, reverse=False)	Sort list in place
reverse()	Reverse list in place
copy()	Return shallow copy of list

2.1.3 List Comprehensions

```

1 # Basic list comprehension
2 squares = [x**2 for x in range(10)]
3 print(f"Squares: {squares}") # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
4
5 # With condition
6 even_squares = [x**2 for x in range(10) if x % 2 == 0]
7 print(f"Even squares: {even_squares}") # [0, 4, 16, 36, 64]
8
9 # With if-else
10 parity = ["even" if x % 2 == 0 else "odd" for x in range(5)]
11 print(f"Parity: {parity}") # ['even', 'odd', 'even', 'odd', 'even']
12
13 # Nested comprehensions
14 matrix = [[i * j for j in range(1, 4)] for i in range(1, 4)]
15 print(f"Matrix: {matrix}") # [[1, 2, 3], [2, 4, 6], [3, 6, 9]]
16
17 # Flatten nested list
18 nested = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
19 flattened = [num for row in nested for num in row]
20 print(f"Flattened: {flattened}") # [1, 2, 3, 4, 5, 6, 7, 8, 9]
21
22 # Practical examples
23 names = ["Alice", "Bob", "Charlie", "Diana"]
24 upper_names = [name.upper() for name in names]
25 name_lengths = [len(name) for name in names]
26 a_names = [name for name in names if name.startswith('A')]

```

```

27 print(f"Upper: {upper_names}")
28 print(f"Lengths: {name_lengths}")
29 print(f"A-names: {a_names}")

```

Listing 2.2: List Comprehensions

2.1.4 Advanced List Operations

```

1 # Sorting
2 numbers = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
3 print(f"Sorted: {sorted(numbers)}")          # New sorted list
4 numbers.sort()                               # In-place sort
5 print(f"Sorted in-place: {numbers}")
6
7 # Custom sorting
8 students = [
9     {"name": "Alice", "grade": 85},
10    {"name": "Bob", "grade": 92},
11    {"name": "Charlie", "grade": 78}
12 ]
13 sorted_by_grade = sorted(students, key=lambda s: s["grade"], reverse=
    True)
14 print(f"Sorted by grade: {sorted_by_grade}")
15
16 # enumerate - get index and value
17 for i, fruit in enumerate(["apple", "banana", "cherry"]):
18     print(f"{i}: {fruit}")
19
20 # zip - combine lists
21 names = ["Alice", "Bob", "Charlie"]
22 ages = [25, 30, 35]
23 cities = ["NY", "LA", "CHI"]
24 for name, age, city in zip(names, ages, cities):
25     print(f"{name} is {age} from {city}")
26
27 # filter and map
28 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
29 even = list(filter(lambda x: x % 2 == 0, numbers))
30 doubled = list(map(lambda x: x * 2, numbers))
31 print(f"Even: {even}")
32 print(f"Doubled: {doubled}")
33
34 # any and all
35 values = [True, True, False]
36 print(f"Any True: {any(values)}")           # True
37 print(f"All True: {all(values)}")           # False
38
39 # Copying lists (shallow vs deep)
40 original = [1, [2, 3], 4]
41 shallow = original.copy()                   # or list(original) or original[:]
42 import copy
43 deep = copy.deepcopy(original)
44
45 original[1][0] = 99
46 print(f"Original: {original}")               # [1, [99, 3], 4]
47 print(f"Shallow: {shallow}")                 # [1, [99, 3], 4] (affected!)

```

```
48 print(f"Deep: {deep}") # [1, [2, 3], 4] (not affected)
```

Listing 2.3: Advanced List Operations

2.2 Tuples

Tuples

Tuples are immutable sequences, typically used to store collections of heterogeneous data. Once created, they cannot be modified (no adding, removing, or changing elements).

```
1 # Creating tuples
2 empty_tuple = ()
3 single_element = (1,) # Comma required!
4 numbers = (1, 2, 3, 4, 5)
5 mixed = (1, "hello", 3.14)
6
7 # Without parentheses (tuple packing)
8 coordinates = 10, 20, 30
9 print(f"Type: {type(coordinates)}") # <class 'tuple'>
10
11 # Accessing (same as lists)
12 point = (3, 4)
13 print(f"x: {point[0]}, y: {point[1]}")
14
15 # Tuple unpacking
16 x, y = point
17 print(f"x={x}, y={y}")
18
19 # Multiple assignment with tuples
20 a, b, c = 1, 2, 3
21
22 # Swapping with tuples
23 a, b = 5, 10
24 a, b = b, a
25 print(f"Swapped: a={a}, b={b}")
26
27 # Tuple methods
28 numbers = (1, 2, 3, 2, 4, 2, 5)
29 print(f"Count of 2: {numbers.count(2)}") # 3
30 print(f"Index of 3: {numbers.index(3)}") # 2
31
32 # Named tuples (more readable)
33 from collections import namedtuple
34
35 Point = namedtuple("Point", ["x", "y"])
36 p1 = Point(3, 4)
37 p2 = Point(x=5, y=6)
38 print(f"p1: ({p1.x}, {p1.y})")
39 print(f"p2: ({p2[0]}, {p2[1]})") # Can also use index
40
41 # Tuples as dictionary keys (lists can't be keys!)
42 locations = {
43     (40.7128, -74.0060): "New York",
```

```

44     (34.0522, -118.2437): "Los Angeles",
45 }
46 print(f"NYC: {locations[(40.7128, -74.0060)]}")

```

Listing 2.4: Tuple Operations

2.3 Dictionaries

Dictionaries

Dictionaries are unordered (ordered in Python 3.7+), mutable collections of key-value pairs. Keys must be hashable (immutable) and unique.

```

1 # Creating dictionaries
2 empty_dict = {}
3 person = {
4     "name": "Alice Johnson",
5     "age": 25,
6     "city": "New York",
7     "email": "alice@example.com"
8 }
9
10 # Using dict() constructor
11 person2 = dict(name="Bob Smith", age=30, city="Los Angeles")
12
13 # Accessing values
14 print(f"Name: {person['name']}")
15 print(f"Age: {person.get('age')}")
16 print(f"Phone: {person.get('phone', 'Not provided')}") # Default value
17
18 # Modifying
19 person["age"] = 26 # Update existing
20 person["phone"] = "555-0123" # Add new
21 person.update({"city": "Boston", "state": "MA"}) # Update multiple
22 print(f"Updated: {person}")
23
24 # Removing
25 removed = person.pop("phone") # Remove and return
26 del person["state"] # Delete
27 person.popitem() # Remove last item (Python 3.7+)
28 person.clear() # Remove all
29
30 # Dictionary views
31 person = {"name": "Alice", "age": 25, "city": "NY"}
32 print(f"Keys: {list(person.keys())}")
33 print(f"Values: {list(person.values())}")
34 print(f"Items: {list(person.items())}")
35
36 # Iterating
37 for key in person:
38     print(f"{key}: {person[key]}")
39
40 for key, value in person.items():
41     print(f"{key} -> {value}")
42
43 # Dictionary comprehensions

```

```

44 squares = {x: x**2 for x in range(5)}
45 print(f"Squares dict: {squares}") # {0:0, 1:1, 2:4, 3:9, 4:16}
46
47 # Merging dictionaries (Python 3.5+)
48 dict1 = {"a": 1, "b": 2}
49 dict2 = {"b": 3, "c": 4}
50 merged = {**dict1, **dict2} # dict2 overwrites dict1
51 print(f"Merged: {merged}") # {'a': 1, 'b': 3, 'c': 4}

```

Listing 2.5: Dictionary Operations

2.3.1 Advanced Dictionary Types

```

1 from collections import defaultdict, Counter, OrderedDict
2
3 # defaultdict - provides default value for missing keys
4 word_count = defaultdict(int)
5 words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
6 for word in words:
7     word_count[word] += 1
8 print(f"Word count: {dict(word_count)}")
9
10 # Counter - specialized for counting
11 counter = Counter(words)
12 print(f"Counter: {counter}")
13 print(f"Most common: {counter.most_common(2)}")
14
15 # OrderedDict - remembers insertion order (less needed in Python 3.7+)
16 ordered = OrderedDict()
17 ordered["first"] = 1
18 ordered["second"] = 2
19 ordered["third"] = 3
20 print(f"Ordered: {ordered}")
21
22 # ChainMap - search multiple dictionaries
23 from collections import ChainMap
24 defaults = {"color": "red", "size": "medium"}
25 user_settings = {"color": "blue"}
26 settings = ChainMap(user_settings, defaults)
27 print(f"Color: {settings['color']}") # blue (from user)
28 print(f"Size: {settings['size']}") # medium (from defaults)

```

Listing 2.6: Advanced Dictionary Types

2.4 Sets

Sets

Sets are unordered collections of unique, hashable elements. They support mathematical set operations like union, intersection, and difference.

```

1 # Creating sets
2 empty_set = set() # Not {} (that's a dict!)
3 numbers = {1, 2, 3, 4, 5}

```

```

4 from_list = set([1, 2, 2, 3, 3, 3]) # {1, 2, 3} - duplicates removed!
5
6 # Adding and removing
7 numbers.add(6)
8 numbers.add(3) # No effect (already exists)
9 numbers.remove(1) # Raises KeyError if not found
10 numbers.discard(10) # No error if not found
11 popped = numbers.pop() # Remove and return arbitrary element
12 print(f"After operations: {numbers}")
13
14 # Set operations
15 a = {1, 2, 3, 4, 5}
16 b = {4, 5, 6, 7, 8}
17
18 print(f"Union: {a | b}") # {1, 2, 3, 4, 5, 6, 7, 8}
19 print(f"Intersection: {a & b}") # {4, 5}
20 print(f"Difference: {a - b}") # {1, 2, 3}
21 print(f"Symmetric Diff: {a ^ b}") # {1, 2, 3, 6, 7, 8}
22
23 # Set methods
24 print(f"a.union(b): {a.union(b)}")
25 print(f"a.intersection(b): {a.intersection(b)}")
26 print(f"a.issubset(b): {a.issubset(b)}")
27 print(f"a.issuperset({4,5}): {a.issuperset({4, 5})}")
28 print(f"a.isdisjoint({6,7}): {a.isdisjoint({6, 7})}")
29
30 # Set comprehensions
31 squares_set = {x**2 for x in range(10)}
32 print(f"Squares set: {squares_set}")
33
34 # Practical use: Removing duplicates
35 items = [1, 2, 2, 3, 3, 3, 4, 5, 5]
36 unique = list(set(items))
37 print(f"Unique items: {unique}")
38
39 # Frozen sets (immutable sets)
40 frozen = frozenset([1, 2, 3])
41 # frozen.add(4) # Error! Immutable

```

Listing 2.7: Set Operations

Chapter 3

Arrays and Matrices in Python

Book Definitions

Source: Python Official Documentation

"The `array` module provides an array object that is like a list but stores only homogeneous data and stores it more compactly. For numerical computing, NumPy arrays provide powerful N-dimensional array objects and sophisticated broadcasting capabilities."

3.1 Python's array Module

```
1 from array import array
2
3 # Creating arrays (type code specified)
4 # 'i' = signed integer, 'f' = float, 'd' = double
5 int_array = array('i', [1, 2, 3, 4, 5])
6 float_array = array('f', [1.0, 2.5, 3.7])
7
8 print(f"Int array: {int_array}")
9 print(f"Float array: {float_array}")
10 print(f"Type: {type(int_array)}")
11
12 # Array operations
13 int_array.append(6)
14 int_array.extend([7, 8, 9])
15 int_array.insert(0, 0)
16 print(f"After adds: {int_array}")
17
18 # Accessing (like lists)
19 print(f"First: {int_array[0]}")
20 print(f"Slice: {int_array[2:5]}")
21
22 # Converting between array and list
23 to_list = int_array.tolist()
24 from_list = array('i', to_list)
25
26 # Array type codes
27 # 'b' = signed char, 'B' = unsigned char
28 # 'i' = signed int, 'I' = unsigned int
29 # 'f' = float, 'd' = double
```

```

30 bytes_array = array('b', [65, 66, 67])
31 print(f"Bytes as chars: {bytes_array.tobytes()}") # b'ABC'

```

Listing 3.1: Using the array Module

3.2 Lists as Matrices (Nested Lists)

```

1 # Creating matrices as nested lists
2 matrix_a = [
3     [1, 2, 3],
4     [4, 5, 6],
5     [7, 8, 9]
6 ]
7
8 matrix_b = [
9     [9, 8, 7],
10    [6, 5, 4],
11    [3, 2, 1]
12 ]
13
14 # Accessing elements
15 print(f"Element [1][1]: {matrix_a[1][1]}") # 5
16 print(f"Row 0: {matrix_a[0]}")
17 print(f"Column 0: {[row[0] for row in matrix_a]}")
18
19 # Matrix addition
20 def matrix_add(m1, m2):
21     """Add two matrices of same dimensions"""
22     rows, cols = len(m1), len(m1[0])
23     return [[m1[i][j] + m2[i][j] for j in range(cols)]
24             for i in range(rows)]
25
26 result_add = matrix_add(matrix_a, matrix_b)
27 print(f"Matrix Addition:")
28 for row in result_add:
29     print(row)
30
31 # Matrix multiplication
32 def matrix_multiply(m1, m2):
33     """Multiply two matrices"""
34     rows_m1, cols_m1 = len(m1), len(m1[0])
35     rows_m2, cols_m2 = len(m2), len(m2[0])
36
37     if cols_m1 != rows_m2:
38         raise ValueError("Incompatible dimensions")
39
40     result = [[0 for _ in range(cols_m2)] for _ in range(rows_m1)]
41     for i in range(rows_m1):
42         for j in range(cols_m2):
43             for k in range(cols_m1):
44                 result[i][j] += m1[i][k] * m2[k][j]
45     return result
46
47 result_mult = matrix_multiply(matrix_a, matrix_b)
48 print(f"\nMatrix Multiplication:")
49 for row in result_mult:

```

```

50     print(row)
51
52 # Matrix transpose
53 def transpose(matrix):
54     """Transpose a matrix"""
55     return [[matrix[j][i] for j in range(len(matrix))]
56             for i in range(len(matrix[0]))]
57
58 print(f"\nTranspose of A:")
59 for row in transpose(matrix_a):
60     print(row)
61
62 # Identity matrix
63 def identity(n):
64     return [[1 if i == j else 0 for j in range(n)] for i in range(n)]
65
66 print(f"\n3x3 Identity:")
67 for row in identity(3):
68     print(row)
69
70 # Scalar multiplication
71 def scalar_multiply(matrix, scalar):
72     return [[elem * scalar for elem in row] for row in matrix]
73
74 print(f"\nA * 2:")
75 for row in scalar_multiply(matrix_a, 2):
76     print(row)

```

Listing 3.2: Working with Matrices using Lists

Chapter 4

NumPy - Numerical Python

Book Definitions

Source: NumPy Official Documentation

"NumPy is the fundamental package for scientific computing in Python. It provides a powerful N-dimensional array object, sophisticated broadcasting functions, tools for integrating C/C++/Fortran code, and useful linear algebra, Fourier transform, and random number capabilities."

4.1 NumPy Arrays

```
1 import numpy as np
2
3 # Creating arrays
4 arr1d = np.array([1, 2, 3, 4, 5])
5 arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
6 arr3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
7
8 print(f"1D Array:\n{arr1d}")
9 print(f"Shape: {arr1d.shape}")           # (5,)
10 print(f"Dimensions: {arr1d.ndim}")       # 1
11 print(f>Data type: {arr1d.dtype}")       # int64
12 print(f"Size: {arr1d.size}")             # 5
13
14 print(f"\n2D Array:\n{arr2d}")
15 print(f"Shape: {arr2d.shape}")           # (3, 3)
16
17 print(f"\n3D Array:\n{arr3d}")
18 print(f"Shape: {arr3d.shape}")           # (2, 2, 2)
19
20 # Array creation functions
21 zeros = np.zeros((3, 4))
22 ones = np.ones((2, 3))
23 identity = np.eye(4)
24 random_arr = np.random.rand(3, 3)
25 random_int = np.random.randint(0, 10, (3, 3))
26 range_arr = np.arange(0, 10, 2)          # Start, stop, step
27 linspace = np.linspace(0, 1, 5)         # 5 evenly spaced numbers
28
29 print(f"Zeros:\n{zeros}")
```

```

30 print(f"Ones:\n{ones}")
31 print(f"Arange: {range_arr}")
32 print(f"Linspace: {linspace}")

```

Listing 4.1: NumPy Array Creation and Basics

4.2 NumPy Array Operations

```

1 import numpy as np
2
3 a = np.array([[1, 2, 3], [4, 5, 6]])
4 b = np.array([[7, 8, 9], [10, 11, 12]])
5
6 # Element-wise operations
7 print(f"a + b:\n{a + b}")
8 print(f"a * b:\n{a * b}")           # Element-wise multiplication
9 print(f"a * 2:\n{a * 2}")           # Scalar multiplication
10 print(f"a ** 2:\n{a ** 2}")         # Square
11 print(f"np.sqrt(a):\n{np.sqrt(a)}")
12
13 # Matrix operations
14 print(f"Transpose:\n{a.T}")
15 print(f"Dot product:\n{np.dot(a, b.T)}")
16 print(f"Matrix multiplication:\n{a @ b.T}")
17
18 # Statistical operations
19 data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
20 print(f"Mean: {np.mean(data)}")
21 print(f"Median: {np.median(data)}")
22 print(f"Std: {np.std(data)}")
23 print(f"Min: {np.min(data)}, Max: {np.max(data)}")
24 print(f"Sum: {np.sum(data)}")
25 print(f"Percentile 50: {np.percentile(data, 50)}")
26
27 # Axis operations
28 matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
29 print(f"Sum of rows (axis=0): {np.sum(matrix, axis=0)}")
30 print(f"Sum of columns (axis=1): {np.sum(matrix, axis=1)}")

```

Listing 4.2: NumPy Array Operations

4.3 Indexing and Slicing

```

1 import numpy as np
2
3 arr = np.array([[1, 2, 3, 4],
4                 [5, 6, 7, 8],
5                 [9, 10, 11, 12],
6                 [13, 14, 15, 16]])
7
8 # Basic indexing
9 print(f"Element [0,0]: {arr[0, 0]}")   # 1
10 print(f"Element [1,2]: {arr[1, 2]}")   # 7
11

```

```

12 # Slicing
13 print(f"First two rows:\n{arr[:2]}")
14 print(f"Last two columns:\n{arr[:, -2:]}")
15 print(f"Sub-matrix:\n{arr[1:3, 1:3]}")
16
17 # Boolean indexing
18 print(f"Elements > 10:\n{arr[arr > 10]}")
19 print(f"Even elements:\n{arr[arr % 2 == 0]}")
20
21 # Fancy indexing (using arrays of indices)
22 rows = np.array([0, 2])
23 cols = np.array([1, 3])
24 print(f"Fancy indexing:\n{arr[rows[:, np.newaxis], cols]}")
25
26 # Conditional selection
27 arr_copy = arr.copy()
28 arr_copy[arr_copy < 5] = 0
29 print(f"Where < 5 -> 0:\n{arr_copy}")
30
31 # np.where
32 result = np.where(arr > 10, "High", "Low")
33 print(f"Where > 10:\n{result}")

```

Listing 4.3: NumPy Indexing and Slicing

4.4 Reshaping and Stacking

```

1 import numpy as np
2
3 arr = np.arange(12)
4 print(f"Original: {arr}")
5
6 # Reshape
7 print(f"2x6:\n{arr.reshape(2, 6)}")
8 print(f"3x4:\n{arr.reshape(3, 4)}")
9 print(f"4x3:\n{arr.reshape(4, 3)}")
10
11 # Flatten
12 matrix = np.array([[1, 2, 3], [4, 5, 6]])
13 print(f"Flatten: {matrix.flatten()}") # Copy
14 print(f"Ravel: {matrix.ravel()}") # View when possible
15
16 # Transpose
17 print(f"Original shape: {matrix.shape}")
18 print(f"Transpose shape: {matrix.T.shape}")
19
20 # Stacking arrays
21 a = np.array([[1, 2], [3, 4]])
22 b = np.array([[5, 6], [7, 8]])
23
24 print(f"Vertical stack:\n{np.vstack((a, b))}")
25 print(f"Horizontal stack:\n{np.hstack((a, b))}")
26 print(f"Depth stack:\n{np.dstack((a, b))}")
27
28 # Splitting arrays
29 arr = np.arange(10)

```

```

30 print(f"Split into 5: {np.split(arr, 5)}")
31 print(f"Split at indices: {np.split(arr, [3, 7])}")

```

Listing 4.4: Reshaping Arrays

4.5 Linear Algebra with NumPy

```

1 import numpy as np
2 from numpy import linalg
3
4 # Create matrices
5 A = np.array([[1, 2], [3, 4]])
6 B = np.array([[5, 6], [7, 8]])
7 v = np.array([1, 2])
8
9 # Matrix operations
10 print(f"Matrix multiplication:\n{A @ B}")
11 print(f"Matrix-vector multiplication: {A @ v}")
12
13 # Determinant
14 print(f"Determinant of A: {linalg.det(A)}")
15
16 # Inverse
17 A_inv = linalg.inv(A)
18 print(f"Inverse of A:\n{A_inv}")
19 print(f"A @ A_inv:\n{A @ A_inv}") # Should be identity
20
21 # Eigenvalues and eigenvectors
22 eigenvalues, eigenvectors = linalg.eig(A)
23 print(f"Eigenvalues: {eigenvalues}")
24 print(f"Eigenvectors:\n{eigenvectors}")
25
26 # Solving linear equations: Ax = b
27 b = np.array([5, 11])
28 x = linalg.solve(A, b)
29 print(f"Solution to Ax = b: {x}")
30 print(f"Verify A @ x = b: {A @ x}")
31
32 # Norms
33 print(f"L1 norm: {linalg.norm(A, 1)}")
34 print(f"L2 norm: {linalg.norm(A, 2)}")
35 print(f"Frobenius norm: {linalg.norm(A, 'fro')}")
36
37 # SVD (Singular Value Decomposition)
38 U, S, Vt = linalg.svd(A)
39 print(f"Singular values: {S}")

```

Listing 4.5: Linear Algebra Operations

Chapter 5

Pandas - Data Analysis Library

Book Definitions

Source: Pandas Official Documentation

"pandas is a fast, powerful, flexible, and easy to use open source data analysis and manipulation tool, built on top of the Python programming language. It provides data structures like Series and DataFrame for working with structured (tabular, multidimensional, potentially heterogeneous) and time series data."

5.1 Pandas Series

```
1 import pandas as pd
2 import numpy as np
3
4 # Creating Series
5 s1 = pd.Series([1, 2, 3, 4, 5])
6 s2 = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
7 s3 = pd.Series({'x': 100, 'y': 200, 'z': 300})
8
9 print(f"Series with default index:\n{s1}")
10 print(f"\nSeries with custom index:\n{s2}")
11 print(f"\nSeries from dict:\n{s3}")
12
13 # Accessing elements
14 print(f"s2['b']: {s2['b']}")
15 print(f"s2[0:2]:\n{s2[0:2]}")
16
17 # Series operations
18 print(f"Element-wise operations:\n{s1 * 2}")
19 print(f"Filtering:\n{s1[s1 > 3]}")
20
21 # Series methods
22 data = pd.Series([1, 2, 3, 4, 5, np.nan, 7, 8, 9, 10])
23 print(f"Mean (excluding NaN): {data.mean()}")
24 print(f"Sum: {data.sum()}")
25 print(f"Count non-NaN: {data.count()}")
26 print(f"Describe:\n{data.describe()}")
```

Listing 5.1: Pandas Series

5.2 Pandas DataFrame

```
1 import pandas as pd
2 import numpy as np
3
4 # Creating DataFrames
5 # From dictionary
6 df1 = pd.DataFrame({
7     'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],
8     'Age': [25, 30, 35, 28],
9     'City': ['NY', 'LA', 'CHI', 'HOU'],
10    'Salary': [50000, 60000, 75000, 55000]
11 })
12
13 # From list of lists
14 df2 = pd.DataFrame([
15     [1, 2, 3],
16     [4, 5, 6],
17     [7, 8, 9]
18 ], columns=['A', 'B', 'C'], index=['row1', 'row2', 'row3'])
19
20 # From NumPy array
21 df3 = pd.DataFrame(
22     np.random.randn(5, 4),
23     columns=['Col1', 'Col2', 'Col3', 'Col4']
24 )
25
26 print(f"DataFrame 1:\n{df1}")
27 print(f"\nDataFrame 2:\n{df2}")
28
29 # DataFrame info
30 print(f"\nShape: {df1.shape}")
31 print(f"Columns: {df1.columns.tolist()}")
32 print(f"Index: {df1.index.tolist()}")
33 print(f"Data types:\n{df1.dtypes}")
34 print(f"Info:")
35 df1.info()
36 print(f"\nDescribe:\n{df1.describe()}")
```

Listing 5.2: DataFrame Creation and Basics

5.3 DataFrame Operations

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
5     'Age': [25, 30, 35, 28, 22],
6     'City': ['NY', 'LA', 'CHI', 'HOU', 'SF'],
7     'Salary': [50000, 60000, 75000, 55000, 48000]
8 })
9
10 # Column selection
11 print(f"Single column:\n{df['Name']}")
12 print(f"\nMultiple columns:\n{df[['Name', 'Salary']]}")
13
```

```

14 # Row selection
15 print(f"\nFirst 3 rows:\n{df.head(3)}")
16 print(f"\nLast 2 rows:\n{df.tail(2)}")
17
18 # iloc - integer-based indexing
19 print(f"\niloc[0]: {df.iloc[0].to_dict()}")
20 print(f"\niloc[1:3]:\n{df.iloc[1:3]}")
21 print(f"\niloc[:, [0, 2]]:\n{df.iloc[:, [0, 2]]}")
22
23 # loc - label-based indexing
24 print(f"\nloc[2]:\n{df.loc[2]}")
25 print(f"\nloc[df['Age'] > 25, ['Name', 'Age']:\n{df.loc[df['Age'] >
    25, ['Name', 'Age']]}")
26
27 # Boolean filtering
28 high_salary = df[df['Salary'] > 55000]
29 print(f"\nHigh salary:\n{high_salary}")
30
31 ny_residents = df[df['City'] == 'NY']
32 print(f"\nNY residents:\n{ny_residents}")
33
34 # Multiple conditions
35 young_high_salary = df[(df['Age'] < 30) & (df['Salary'] > 50000)]
36 print(f"\nYoung and high salary:\n{young_high_salary}")

```

Listing 5.3: DataFrame Indexing and Selection

5.4 Data Manipulation

```

1 import pandas as pd
2
3 df = pd.DataFrame({
4     'Name': ['Alice', 'Bob', 'Charlie'],
5     'Age': [25, 30, 35],
6     'Salary': [50000, 60000, 75000]
7 })
8
9 # Adding columns
10 df['Department'] = ['HR', 'IT', 'Finance']
11 df['Bonus'] = df['Salary'] * 0.1
12 df['Total'] = df['Salary'] + df['Bonus']
13
14 # Adding rows
15 new_row = pd.DataFrame({
16     'Name': ['Diana'],
17     'Age': [28],
18     'Salary': [55000],
19     'Department': ['Marketing'],
20     'Bonus': [5500],
21     'Total': [60500]
22 })
23 df = pd.concat([df, new_row], ignore_index=True)
24
25 print(f"After adding:\n{df}")
26
27 # Renaming columns

```

```

28 df = df.rename(columns={'Salary': 'Base_Salary', 'Total': '
    Total_Compensation'})
29 print(f"\nRenamed columns:\n{df.columns.tolist()}")
30
31 # Dropping columns and rows
32 df_dropped_col = df.drop('Bonus', axis=1)
33 df_dropped_row = df.drop(0, axis=0)
34
35 # Applying functions
36 df['Age_Group'] = df['Age'].apply(lambda x: 'Young' if x < 30 else '
    Senior')
37 df['Name_Length'] = df['Name'].str.len()
38
39 print(f"\nAfter apply:\n{df}")
40
41 # Sorting
42 df_sorted = df.sort_values('Base_Salary', ascending=False)
43 print(f"\nSorted by salary:\n{df_sorted}")

```

Listing 5.4: Adding, Removing, and Modifying Data

5.5 Grouping and Aggregation

```

1 import pandas as pd
2 import numpy as np
3
4 df = pd.DataFrame({
5     'Department': ['IT', 'IT', 'HR', 'HR', 'Finance', 'Finance', 'IT'],
6     'Employee': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', '
    Grace'],
7     'Salary': [50000, 60000, 45000, 48000, 75000, 70000, 55000],
8     'Experience': [3, 5, 2, 4, 8, 6, 3]
9 })
10
11 print(f"Original data:\n{df}")
12
13 # Group by single column
14 dept_group = df.groupby('Department')
15 print(f"\nMean salary by department:\n{dept_group['Salary'].mean()}")
16 print(f"\nSum salary by department:\n{dept_group['Salary'].sum()}")
17 print(f"\nCount by department:\n{dept_group.size()}")
18
19 # Multiple aggregations
20 agg_results = dept_group.agg({
21     'Salary': ['mean', 'min', 'max', 'sum'],
22     'Experience': ['mean', 'max']
23 })
24 print(f"\nMultiple aggregations:\n{agg_results}")
25
26 # Custom aggregation
27 def salary_range(series):
28     return series.max() - series.min()
29
30 custom_agg = dept_group['Salary'].agg(['mean', 'std', salary_range])
31 print(f"\nCustom aggregation:\n{custom_agg}")
32

```

```

33 # Filtering groups
34 high_paying_depts = dept_group.filter(lambda x: x['Salary'].mean() >
    50000)
35 print(f"\nHigh paying departments:\n{high_paying_depts}")

```

Listing 5.5: GroupBy Operations

5.6 Handling Missing Data

```

1 import pandas as pd
2 import numpy as np
3
4 # Create DataFrame with missing values
5 df = pd.DataFrame({
6     'A': [1, 2, np.nan, 4, 5],
7     'B': [np.nan, 2, 3, np.nan, 5],
8     'C': [1, 2, 3, 4, 5],
9     'D': ['a', 'b', np.nan, 'd', 'e']
10 })
11
12 print(f"Original data:\n{df}")
13
14 # Detecting missing values
15 print(f"\nIs null:\n{df.isnull()}")
16 print(f"\nNull count:\n{df.isnull().sum()}")
17 print(f"\nNot null count:\n{df.notnull().sum()}")
18
19 # Dropping missing values
20 print(f"\nDrop rows with any NaN:\n{df.dropna()}")
21 print(f"\nDrop rows with all NaN:\n{df.dropna(how='all')}")
22 print(f"\nDrop columns with any NaN:\n{df.dropna(axis=1)}")
23
24 # Filling missing values
25 print(f"\nFill with 0:\n{df.fillna(0)}")
26 print(f"\nFill with mean:\n{df.fillna({'A': df['A'].mean(), 'B': df['B']
    '.mean()})}")
27 print(f"\nForward fill:\n{df.fillna(method='ffill')}")
28 print(f"\nBackward fill:\n{df.fillna(method='bfill')}")
29
30 # Interpolation
31 df_numeric = df.select_dtypes(include=[np.number])
32 print(f"\nInterpolate:\n{df_numeric.interpolate()}")

```

Listing 5.6: Missing Data Operations

5.7 Merging and Joining DataFrames

```

1 import pandas as pd
2
3 # Create DataFrames
4 employees = pd.DataFrame({
5     'emp_id': [1, 2, 3, 4],
6     'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
7     'dept_id': [101, 102, 101, 103]

```

```

8 })
9
10 departments = pd.DataFrame({
11     'dept_id': [101, 102, 103],
12     'dept_name': ['IT', 'HR', 'Finance']
13 })
14
15 salaries = pd.DataFrame({
16     'emp_id': [1, 2, 3, 4],
17     'salary': [50000, 60000, 75000, 55000],
18     'bonus': [5000, 6000, 7500, 5500]
19 })
20
21 print(f"Employees:\n{employees}")
22 print(f"\nDepartments:\n{departments}")
23 print(f"\nSalaries:\n{salaries}")
24
25 # Inner join
26 inner_join = pd.merge(employees, departments, on='dept_id', how='inner',
27 )
28 print(f"\nInner join:\n{inner_join}")
29
30 # Left join
31 left_join = pd.merge(employees, departments, on='dept_id', how='left')
32 print(f"\nLeft join:\n{left_join}")
33
34 # Multiple joins
35 full_data = (employees
36               .merge(departments, on='dept_id')
37               .merge(salaries, on='emp_id'))
38 print(f"\nFull data:\n{full_data}")
39
40 # Concatenation
41 df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
42 df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
43 concatenated = pd.concat([df1, df2], ignore_index=True)
44 print(f"\nConcatenated:\n{concatenated}")

```

Listing 5.7: Merging DataFrames

5.8 Time Series with Pandas

```

1 import pandas as pd
2 import numpy as np
3
4 # Creating time series
5 dates = pd.date_range('2024-01-01', periods=10, freq='D')
6 ts = pd.Series(np.random.randn(10), index=dates)
7 print(f"Time series:\n{ts}")
8
9 # Date range with different frequencies
10 daily = pd.date_range('2024-01-01', periods=5, freq='D')
11 weekly = pd.date_range('2024-01-01', periods=5, freq='W')
12 monthly = pd.date_range('2024-01-01', periods=5, freq='M')
13 business_days = pd.date_range('2024-01-01', periods=5, freq='B')
14

```

```

15 print(f"Daily: {daily}")
16 print(f"Weekly: {weekly}")
17 print(f"Monthly: {monthly}")
18 print(f"Business days: {business_days}")
19
20 # Resampling
21 ts_long = pd.Series(
22     np.random.randn(100),
23     index=pd.date_range('2024-01-01', periods=100, freq='H')
24 )
25 print(f"\nOriginal (hourly): {len(ts_long)} points")
26
27 daily_mean = ts_long.resample('D').mean()
28 print(f"Resampled daily: {len(daily_mean)} points")
29
30 # Shifting
31 print(f"\nOriginal:\n{ts.head()}")
32 print(f"\nShifted by 1:\n{ts.shift(1).head()}")
33
34 # Rolling window
35 rolling_mean = ts.rolling(window=3).mean()
36 print(f"\nRolling mean (window=3):\n{rolling_mean}")

```

Listing 5.8: Time Series Operations

5.9 Input/Output Operations

```

1 import pandas as pd
2
3 # Reading CSV
4 # df = pd.read_csv('data.csv')
5 # df = pd.read_csv('data.csv', sep=';', encoding='utf-8')
6 # df = pd.read_csv('data.csv', usecols=['Name', 'Age'])
7 # df = pd.read_csv('data.csv', nrows=100) # Read first 100 rows
8
9 # Reading Excel
10 # df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
11
12 # Reading JSON
13 # df = pd.read_json('data.json')
14
15 # Writing to files
16 # df.to_csv('output.csv', index=False)
17 # df.to_excel('output.xlsx', sheet_name='Data')
18 # df.to_json('output.json')
19
20 # Example with sample data
21 df = pd.DataFrame({
22     'Name': ['Alice', 'Bob', 'Charlie'],
23     'Age': [25, 30, 35],
24     'Salary': [50000, 60000, 75000]
25 })
26
27 # Export to different formats
28 df.to_csv('sample_output.csv', index=False)
29 print("Data exported to CSV")

```

```

30
31 # Read back
32 df_read = pd.read_csv('sample_output.csv')
33 print(f"Read back:\n{df_read}")
34
35 # Clean up
36 import os
37 os.remove('sample_output.csv')

```

Listing 5.9: Reading and Writing Data

5.10 Pandas Cheat Sheet

Operation	Code
Create DataFrame from dict	pd.DataFrame(dict)
Read CSV	pd.read_csv('file.csv')
Write CSV	df.to_csv('file.csv', index=False)
View first/last rows	df.head(n), df.tail(n)
Select column	df['column']
Select rows by condition	df[df['col'] > value]
Group by	df.groupby('col').mean()
Sort values	df.sort_values('col', ascending=False)
Drop missing	df.dropna()
Fill missing	df.fillna(value)
Apply function	df['col'].apply(func)
Merge DataFrames	pd.merge(df1, df2, on='key')
Concatenate	pd.concat([df1, df2])
Describe statistics	df.describe()
Info about DataFrame	df.info()

5.11 NumPy vs Pandas Comparison

Feature	NumPy	Pandas
Data Structure	ndarray	Series, DataFrame
Data Types	Homogeneous	Heterogeneous
Labels	No labels (axis-based)	Row and column labels
Missing Data	Limited support	Built-in support (NaN)
Operations	Mathematical/linear algebra	Data manipulation/analysis
Performance	Very fast (C-based)	Slightly slower (built on NumPy)
Use Case	Scientific computing	Data analysis, ETL
Size of Data	Large arrays	Tabular data

Chapter 6

Operator Overloading in Python

Book Definitions

Source: "Python Programming" by Reema Thareja

"Operator overloading is a feature of object-oriented programming that allows the same operator to perform different operations depending on the context. In Python, this is achieved by defining special methods (also called magic methods or dunder methods) in a class to handle built-in operators."

Source: Python Official Documentation

"Python allows classes to define their own behavior with respect to language operators. When a class defines a special method, instances of that class can use the standard operators (+, -, *, /, etc.) with objects of that class."

6.1 Simple Explanation

Operator overloading means giving extra meanings to existing operators (like +, -, *, /, etc.) when they work with user-defined objects. It's like teaching Python how to add, subtract, or compare objects of your own class, just like it already knows how to do these operations with numbers.

Think of it this way:

- Python knows $5 + 3 = 8$ (adding numbers)
- Python knows `"Hello" + " World" = "Hello World"` (joining strings)
- But if you create a `Book` or `Student` class, Python doesn't automatically know how to add two books. That's where operator overloading comes in - you teach Python!

6.2 Common Operator Overloading Methods

6.3 Examples

6.3.1 Example 1: Vector Class with Operator Overloading

Operator	Method Name	What it does
+	<code>__add__(self, other)</code>	Addition
-	<code>__sub__(self, other)</code>	Subtraction
*	<code>__mul__(self, other)</code>	Multiplication
/	<code>__truediv__(self, other)</code>	Division
//	<code>__floordiv__(self, other)</code>	Floor Division
%	<code>__mod__(self, other)</code>	Modulus
**	<code>__pow__(self, other)</code>	Power
==	<code>__eq__(self, other)</code>	Equality check
!=	<code>__ne__(self, other)</code>	Not equal
<	<code>__lt__(self, other)</code>	Less than
<=	<code>__le__(self, other)</code>	Less than or equal
>	<code>__gt__(self, other)</code>	Greater than
>=	<code>__ge__(self, other)</code>	Greater than or equal
<code>len()</code>	<code>__len__(self)</code>	Length
<code>str()</code>	<code>__str__(self)</code>	String representation
<code>repr()</code>	<code>__repr__(self)</code>	Official representation
	<code>__getitem__(self, key)</code>	Indexing
=	<code>__setitem__(self, key, value)</code>	Setting item
in	<code>__contains__(self, item)</code>	Membership test
call	<code>__call__(self, *args)</code>	Make instance callable

```

1 class Vector:
2     """2D Vector class with operator overloading"""
3
4     def __init__(self, x, y):
5         self.x = x
6         self.y = y
7
8     # Arithmetic operators
9     def __add__(self, other):
10        """Vector addition"""
11        if isinstance(other, Vector):
12            return Vector(self.x + other.x, self.y + other.y)
13        raise TypeError(f"Cannot add Vector and {type(other).__name__}")
14    )
15
16    def __sub__(self, other):
17        """Vector subtraction"""
18        if isinstance(other, Vector):
19            return Vector(self.x - other.x, self.y - other.y)
20        raise TypeError(f"Cannot subtract {type(other).__name__} from Vector")
21
22    def __mul__(self, scalar):
23        """Scalar multiplication"""
24        if isinstance(scalar, (int, float)):
25            return Vector(self.x * scalar, self.y * scalar)
26        raise TypeError(f"Cannot multiply Vector by {type(scalar).__name__}")
27
28    def __rmul__(self, scalar):
29        """Right scalar multiplication (scalar * Vector)"""

```

```

29         return self.__mul__(scalar)
30
31     def __truediv__(self, scalar):
32         """Scalar division"""
33         if isinstance(scalar, (int, float)) and scalar != 0:
34             return Vector(self.x / scalar, self.y / scalar)
35         raise ValueError("Cannot divide by zero or non-numeric type")
36
37     def __neg__(self):
38         """Negation (-Vector)"""
39         return Vector(-self.x, -self.y)
40
41     def __abs__(self):
42         """Absolute value (magnitude)"""
43         return (self.x**2 + self.y**2)**0.5
44
45     # Comparison operators
46     def __eq__(self, other):
47         if isinstance(other, Vector):
48             return self.x == other.x and self.y == other.y
49         return False
50
51     def __lt__(self, other):
52         """Compare by magnitude"""
53         if isinstance(other, Vector):
54             return abs(self) < abs(other)
55         return NotImplemented
56
57     def __le__(self, other):
58         if isinstance(other, Vector):
59             return abs(self) <= abs(other)
60         return NotImplemented
61
62     # String representation
63     def __str__(self):
64         return f"Vector({self.x}, {self.y})"
65
66     def __repr__(self):
67         return f"Vector(x={self.x}, y={self.y})"
68
69     # Container-like behavior
70     def __len__(self):
71         return 2 # 2D vector has 2 components
72
73     def __getitem__(self, index):
74         if index == 0: return self.x
75         if index == 1: return self.y
76         raise IndexError("Vector index out of range (0-1)")
77
78     def __setitem__(self, index, value):
79         if index == 0: self.x = value
80         elif index == 1: self.y = value
81         else: raise IndexError("Vector index out of range (0-1)")
82
83     def __iter__(self):
84         yield self.x
85         yield self.y
86

```

```

87     def __contains__(self, value):
88         return value == self.x or value == self.y
89
90     # Callable behavior
91     def __call__(self):
92         return (self.x, self.y)
93
94 # Demonstration
95 v1 = Vector(3, 4)
96 v2 = Vector(1, 2)
97 v3 = Vector(3, 4)
98
99 print(f"v1 = {v1}")           # Vector(3, 4)
100 print(f"v2 = {v2}")          # Vector(1, 2)
101 print(f"v1 + v2 = {v1 + v2}") # Vector(4, 6)
102 print(f"v1 - v2 = {v1 - v2}") # Vector(2, 2)
103 print(f"v1 * 3 = {v1 * 3}")   # Vector(9, 12)
104 print(f"3 * v1 = {3 * v1}")   # Vector(9, 12)
105 print(f"v1 / 2 = {v1 / 2}")   # Vector(1.5, 2.0)
106 print(f"-v1 = {-v1}")         # Vector(-3, -4)
107 print(f"|v1| = {abs(v1)}")    # 5.0
108 print(f"v1 == v2: {v1 == v2}") # False
109 print(f"v1 == v3: {v1 == v3}") # True
110 print(f"v1 < v2: {v1 < v2}")  # False (5.0 not < 2.236)
111 print(f"len(v1) = {len(v1)}") # 2
112 print(f"v1[0] = {v1[0]}, v1[1] = {v1[1]}") # 3, 4
113 print(f"3 in v1: {3 in v1}")  # True
114 print(f"v1() = {v1()}")       # (3, 4)
115 print(f"list(v1) = {list(v1)}") # [3, 4]

```

Listing 6.1: Complete Vector Class with Multiple Operators

6.4 Key Points to Remember

- You **cannot create new operators** - you can only overload existing ones
- You **cannot change the operator's arity** - `+` will always be a binary operator
- Use operator overloading **only when it makes intuitive sense** for your class
- **Keep it consistent** - if you overload `+`, make sure it does something that makes sense as "addition" for your objects

Chapter 7

Method Overloading in Python

Book Definitions

Source: "Core Python Programming" by Wesley J. Chun

"Method overloading refers to the ability to define multiple methods with the same name but different parameters within a class. However, unlike languages such as Java or C++, Python does not support traditional method overloading where multiple methods share the same name but have different signatures."

Source: "Learning Python" by Mark Lutz

"In Python, method overloading is achieved through default arguments, variable-length arguments, or by using single dispatch generic functions, rather than through multiple methods with the same name. The last defined method overwrites any previous methods with the same name."

7.1 Simple Explanation

Python handles method overloading differently than other languages - instead of creating multiple methods with the same name, we typically use default arguments or flexible parameter handling to achieve the same result.

Think of it this way:

- In some languages (like Java), you can write: `add(x)`, `add(x, y)`, `add(x, y, z)` - all as separate methods
- In Python, you write just **ONE** method that can handle 1, 2, or 3 parameters using tricks like default values or `*args`

7.2 Techniques for Python Method Overloading

7.3 Example: Using `@singledispatch` for Type-Based Overloading

```
1 from functools import singledispatch
2
3 @singledispatch
```

Technique	Best Used When	Example
Default Arguments	Fixed maximum parameters	<code>def add(a, b=0, c=0)</code>
Variable Arguments (*args)	Unknown number of parameters	<code>def multiply(*args)</code>
Type Checking	Need different behavior based on type	<code>isinstance(arg, str)</code>
Keyword Arguments (**kwargs)	Named parameters	<code>def process(**kwargs)</code>
Single Dispatch	Type-based dispatch	<code>@singledispatch decorator</code>

```

4 def process_data(data):
5     """Default processing for unknown types"""
6     print(f"Default processing: {type(data).__name__} - {data}")
7
8 @process_data.register(str)
9 def _(data):
10    """Process string data"""
11    print(f"Processing string: '{data.upper()}'")
12
13 @process_data.register(int)
14 def _(data):
15    """Process integer data"""
16    print(f"Processing integer: {data} -> doubled: {data * 2}")
17
18 @process_data.register(float)
19 def _(data):
20    """Process float data"""
21    print(f"Processing float: {data:.2f} -> squared: {data**2:.2f}")
22
23 @process_data.register(list)
24 def _(data):
25    """Process list data"""
26    print(f"Processing list: {len(data)} items -> sum: {sum(data)}")
27
28 @process_data.register(dict)
29 def _(data):
30    """Process dictionary data"""
31    print(f"Processing dict: {len(data)} key-value pairs")
32    for k, v in data.items():
33        print(f"    {k}: {v}")
34
35 # Testing different types
36 process_data("hello")           # String
37 process_data(10)                # Integer
38 process_data(3.14)              # Float
39 process_data([1, 2, 3, 4, 5])   # List
40 process_data({"name": "Alice", "age": 30}) # Dictionary
41 process_data(True)              # bool (not specifically registered)

```

Listing 7.1: Type-Based Method Overloading with singledispatch

7.4 Example: Real-World Employee Salary Calculator

```
1 class Employee:
2     def __init__(self, name):
3         self.name = name
4         self.salary = 0
5
6     def calculate_salary(self, *args, **kwargs):
7         """
8         Flexible method to calculate salary in different ways:
9         - calculate_salary(hours, rate) for hourly employees
10        - calculate_salary(monthly_salary) for salaried employees
11        - calculate_salary(base, bonus, commission) for sales people
12        - calculate_salary(hourly_rate=rate, hours_worked=hours) using
13        kwargs
14        """
15        if len(args) == 2: # Hourly employee
16            hours, rate = args
17            self.salary = hours * rate
18            return f"{self.name} (Hourly): ${self.salary:,.2f}"
19
20        elif len(args) == 1: # Salaried employee
21            monthly = args[0]
22            self.salary = monthly
23            return f"{self.name} (Salaried): ${self.salary:,.2f}/month"
24
25        elif len(args) == 3: # Sales person with commission
26            base, bonus, commission = args
27            self.salary = base + bonus + commission
28            return f"{self.name} (Sales): ${self.salary:,.2f} (incl.
29            commission)"
30
31        elif kwargs: # Using keyword arguments
32            if 'hourly_rate' in kwargs and 'hours_worked' in kwargs:
33                self.salary = kwargs['hourly_rate'] * kwargs['
34                hours_worked']
35                return f"{self.name} (Hourly-kwarg): ${self.salary:,.2
36                f}"
37
38        return "Invalid salary calculation method"
39
40 # Testing the Employee class
41 emp1 = Employee("Alice")
42 emp2 = Employee("Bob")
43 emp3 = Employee("Charlie")
44 emp4 = Employee("Diana")
45
46 print(emp1.calculate_salary(40, 25)) # Hourly
47 print(emp2.calculate_salary(5000)) # Salaried
48 print(emp3.calculate_salary(3000, 500, 200)) # Sales with commission
49 print(emp4.calculate_salary(hours_worked=40, hourly_rate=30)) # Using
50 kwargs
```

Listing 7.2: Flexible Method with Multiple Parameter Combinations

Chapter 8

Abstract Base Class (ABC) in Python

Book Definitions

Source: "Python in a Nutshell" by Alex Martelli

"An Abstract Base Class (ABC) is a class that cannot be instantiated and is intended to be subclassed. ABCs define a common interface or blueprint for other classes to follow, often containing abstract methods that must be implemented by concrete subclasses."

Source: "Fluent Python" by Luciano Ramalho

"Abstract Base Classes complement duck typing by providing a way to define interfaces in Python. They allow a class to declare that it provides a particular interface, even without inheritance, through virtual subclasses."

8.1 Simple Explanation

An Abstract Base Class (ABC) is like a **template** or **blueprint** that other classes can follow. It defines *what* methods a subclass must have, but doesn't provide the actual implementation (or may provide partial implementation). You cannot create objects directly from an abstract class - it only exists to be inherited from.

Think of it this way:

- Imagine a blueprint for a "Vehicle" - it says every vehicle must have a `start_engine()` and `stop_engine()` method
- But the blueprint itself isn't a real vehicle - you can't drive a blueprint!
- When you create a "Car" class inheriting from "Vehicle", you must provide the actual implementation of these methods
- This ensures all vehicles have the same basic functionality

8.2 Key ABC Components

8.3 Example: Payment Processing System

Component	Syntax	Purpose
ABC Class	<code>class MyClass(ABC):</code>	Inherit from ABC to create abstract class
<code>@abstractmethod</code>	<code>@abstractmethod</code>	Decorator to mark method as abstract
Abstract Properties	<code>@property @abstractmethod</code>	Abstract property that subclasses must define
Concrete Methods	Regular methods	Provide default implementation
<code>register()</code>	<code>MyABC.register(SubClass)</code>	Register a class as virtual subclass

```

1 from abc import ABC, abstractmethod
2
3 class PaymentProcessor(ABC):
4     """Abstract class for payment processors"""
5
6     def __init__(self, amount):
7         self.amount = amount
8         self.status = "pending"
9
10    @abstractmethod
11    def process_payment(self):
12        """Process the payment - implementation varies by payment
method"""
13        pass
14
15    @abstractmethod
16    def validate_payment(self):
17        """Validate payment details - implementation varies"""
18        pass
19
20    @abstractmethod
21    def generate_receipt(self):
22        """Generate payment receipt"""
23        pass
24
25    # Concrete method - shared across all payment types
26    def display_status(self):
27        """Show payment status"""
28        print(f"Payment Status: {self.status}")
29        print(f"Amount: ${self.amount:,.2f}")
30
31    class CreditCardPayment(PaymentProcessor):
32        def __init__(self, amount, card_number, cvv):
33            super().__init__(amount)
34            self.card_number = card_number
35            self.cvv = cvv
36
37        def validate_payment(self):
38            return len(self.card_number) == 16 and len(self.cvv) == 3
39
40        def process_payment(self):
41            if self.validate_payment():
42                self.status = "completed"
43                return "Credit card payment processed successfully"

```

```

44         else:
45             self.status = "failed"
46             return "Invalid credit card details"
47
48     def generate_receipt(self):
49         last_four = self.card_number[-4:]
50         return f"Receipt: Credit Card - ${self.amount} charged to ...{
last_four}"
51
52 class PayPalPayment(PaymentProcessor):
53     def __init__(self, amount, email):
54         super().__init__(amount)
55         self.email = email
56
57     def validate_payment(self):
58         return "@" in self.email and "." in self.email
59
60     def process_payment(self):
61         if self.validate_payment():
62             self.status = "completed"
63             return "PayPal payment processed successfully"
64         else:
65             self.status = "failed"
66             return "Invalid PayPal email address"
67
68     def generate_receipt(self):
69         return f"Receipt: PayPal - ${self.amount} sent to {self.email}"
70
71 class UPIPayment(PaymentProcessor):
72     def __init__(self, amount, upi_id):
73         super().__init__(amount)
74         self.upi_id = upi_id
75
76     def validate_payment(self):
77         return "@" in self.upi_id and len(self.upi_id) > 5
78
79     def process_payment(self):
80         if self.validate_payment():
81             self.status = "completed"
82             return "UPI payment processed successfully"
83         else:
84             self.status = "failed"
85             return "Invalid UPI ID"
86
87     def generate_receipt(self):
88         return f"Receipt: UPI - ${self.amount} paid via {self.upi_id}"
89
90 # Testing the payment system
91 payments = [
92     CreditCardPayment(150.00, "1234567812345678", "123"),
93     PayPalPayment(75.50, "user@example.com"),
94     UPIPayment(45.25, "user@bank")
95 ]
96
97 for payment in payments:
98     print(f"\n{payment.process_payment()}")
99     payment.display_status()

```

```
print(payment.generate_receipt())
```

Listing 8.1: Complete Payment System using ABC

8.4 Benefits of Using ABCs

- **Enforces Interface:** Ensures all subclasses implement required methods
- **Code Organization:** Creates clear hierarchy and structure
- **Prevents Instantiation:** Cannot create objects from incomplete abstract classes
- **Polymorphism:** Allows treating different objects uniformly through parent class
- **Documentation:** Abstract methods serve as clear documentation of required functionality

Chapter 9

Iterators in Python

Book Definitions

Source: "Python Cookbook" by David Beazley and Brian K. Jones

"An iterator is an object that implements the iterator protocol, consisting of the `__iter__()` and `__next__()` methods. The `__iter__()` method returns the iterator object itself, and the `__next__()` method returns the next item from the container. When no more items are available, `__next__()` raises the `StopIteration` exception."

Source: "Learning Python" by Mark Lutz

"Iterators are objects that represent streams of data. They provide a way to access elements of a container sequentially without exposing the underlying structure."

9.1 Simple Explanation

An iterator is an object that allows you to traverse through a collection of items one at a time. Think of it like a **bookmark** that remembers where you are in a list, tuple, dictionary, or any sequence, and lets you move to the next item whenever you ask for it.

Think of it this way:

- Imagine reading a book with a bookmark
- You start at the first page
- Each time you turn the page, you move to the next page
- When you reach the end, you stop (you can't go further)
- That's exactly what an iterator does with data!

9.2 Iterator vs Iterable

9.3 Example: Custom Fibonacci Iterator

Concept	Definition	Example
Iterable	Object that can return an iterator. Implements <code>__iter__()</code>	List, tuple, string, dictionary
Iterator	Object that remembers its state. Implements <code>__iter__()</code> and <code>__next__()</code>	Result of <code>iter(list)</code>
Can be reused?	Iterables can be iterated multiple times	<code>for i in my_list</code> works multiple times
Exhaustible?	Iterators can only be used once	After <code>list(iterator)</code> , iterator is exhausted

```

1 class FibonacciIterator:
2     """Iterator that generates Fibonacci numbers up to a limit"""
3
4     def __init__(self, limit):
5         self.limit = limit
6         self.a = 0
7         self.b = 1
8         self.count = 0
9
10    def __iter__(self):
11        return self
12
13    def __next__(self):
14        if self.count >= self.limit:
15            raise StopIteration
16
17        if self.count == 0:
18            self.count += 1
19            return self.a
20        elif self.count == 1:
21            self.count += 1
22            return self.b
23        else:
24            self.a, self.b = self.b, self.a + self.b
25            self.count += 1
26            return self.b
27
28 # Using the Fibonacci iterator
29 fib = FibonacciIterator(10)
30 print("First 10 Fibonacci numbers:")
31 for num in fib:
32     print(num, end=" ") # Output: 0 1 1 2 3 5 8 13 21 34
33 print()
34
35 # Convert to list (exhausts the iterator)
36 fib2 = FibonacciIterator(7)
37 print(f"Fibonacci list: {list(fib2)}") # [0, 1, 1, 2, 3, 5, 8]
38 print(f"Second use: {list(fib2)}")    # [] - exhausted!

```

Listing 9.1: Complete Fibonacci Iterator

9.4 Example: Infinite Iterator with islice

```
1 import itertools
2
3 class InfiniteCounter:
4     """Iterator that counts infinitely upward from start"""
5
6     def __init__(self, start=0):
7         self.current = start
8
9     def __iter__(self):
10        return self
11
12    def __next__(self):
13        result = self.current
14        self.current += 1
15        return result
16
17 # Using infinite iterator with a limiting condition
18 counter = InfiniteCounter(10)
19 print("First 5 numbers from infinite counter:")
20 for _ in range(5):
21     print(next(counter), end=" ") # Output: 10 11 12 13 14
22 print()
23
24 # Using itertools.islice to limit infinite iterators
25 counter2 = InfiniteCounter(100)
26 first_8 = list(itertools.islice(counter2, 8))
27 print(f"First 8 numbers: {first_8}") # [100, 101, ..., 107]
```

Listing 9.2: Infinite Counter Iterator

9.5 Common Iterator Patterns

- **Memory Efficient:** Doesn't load all data into memory at once
- **Lazy Evaluation:** Computes values only when needed
- **Unified Interface:** Works with for loops, comprehensions, etc.
- **Infinite Sequences:** Can represent infinite data streams
- **Composable:** Can chain and combine iterators

Chapter 10

Generators in Python

Book Definitions

Source: "Python in a Nutshell" by Alex Martelli

"A generator is a function that produces a sequence of results lazily, meaning it yields values one at a time only when requested. Instead of returning a single value with return, a generator uses the yield keyword to produce a series of values."

Source: "Fluent Python" by Luciano Ramalho

"Generators are functions that use the yield keyword to return values to the caller while maintaining their state between calls. They are a simpler way to create iterators compared to implementing a class with `__iter__` and `__next__` methods."

10.1 Simple Explanation

A generator is a special function that can **pause** its execution and **resume** later. Instead of computing all values at once and returning them (like a regular function), a generator produces values one at a time, on-demand.

Think of it this way:

- A regular function is like a vending machine - you put money in, and it gives you all items at once
- A generator is like a cafeteria line - you get one item at a time, and you can pause between items
- When you want more, you continue from where you left off
- This is extremely memory-efficient for large datasets!

10.2 Generator vs Regular Function vs Iterator Class

10.3 Example: Generator Pipeline for Data Processing

```
1 import itertools
2
3 def read_data_stream():
```

Aspect	Regular Function	Iterator Class	Generator
Returns	Single value	Iterator object	Generator object
Memory	Can be high if returns large list	Low	Very low
State	No state between calls	Maintains state in attributes	Maintains state automatically
Implementation	Simple	Requires <code>__iter__</code> and <code>__next__</code>	Simple with <code>yield</code>
Reusability	Can be called multiple times	One-time use	One-time use

```

4     """Simulates a continuous data stream"""
5     for i in itertools.count(1):
6         yield {"id": i, "value": i * 10}
7         # In real scenario, might read from network/file
8
9 def filter_by_threshold(source, threshold):
10     """Filter data based on value threshold"""
11     for item in source:
12         if item["value"] > threshold:
13             yield item
14
15 def transform_data(source):
16     """Transform data by doubling values"""
17     for item in source:
18         item["value"] = item["value"] * 2
19         item["processed"] = True
20         yield item
21
22 def batch_processor(source, batch_size):
23     """Group items into batches"""
24     batch = []
25     for item in source:
26         batch.append(item)
27         if len(batch) >= batch_size:
28             yield batch
29             batch = []
30     if batch: # Yield remaining items
31         yield batch
32
33 # Create the generator pipeline
34 data_stream = read_data_stream()
35 filtered_stream = filter_by_threshold(data_stream, 50)
36 transformed_stream = transform_data(filtered_stream)
37 batched_stream = batch_processor(transformed_stream, 3)
38
39 # Process first 5 batches
40 print("Processing data pipeline:")
41 for i, batch in enumerate(itertools.islice(batched_stream, 5)):
42     print(f"Batch {i+1}: {batch}")

```

Listing 10.1: Generator Pipeline for Data Processing

10.4 Example: Generator with yield from (Delegation)

```
1 class TreeNode:
2     def __init__(self, value, children=None):
3         self.value = value
4         self.children = children or []
5
6     def __iter__(self):
7         """Iterate over tree using yield from"""
8         yield self.value
9         for child in self.children:
10             yield from child # Delegate to child's iterator
11
12     def add_child(self, child):
13         self.children.append(child)
14         return child
15
16 # Create a tree structure
17 root = TreeNode(1)
18 child1 = root.add_child(TreeNode(2))
19 child2 = root.add_child(TreeNode(3))
20 child1.add_child(TreeNode(4))
21 child1.add_child(TreeNode(5))
22 child2.add_child(TreeNode(6))
23 child2.add_child(TreeNode(7))
24
25 print("Tree traversal (depth-first):")
26 for value in root:
27     print(value, end=" ") # Output: 1 2 4 5 3 6 7
28 print()
29
30 # Generator function using yield from for flattening
31 def flatten(nested_list):
32     """Generator that flattens nested lists using yield from"""
33     for item in nested_list:
34         if isinstance(item, list):
35             yield from flatten(item) # Recursively flatten
36         else:
37             yield item
38
39 nested = [1, [2, 3], [4, [5, 6]], 7]
40 flattened = list(flatten(nested))
41 print(f"Flattened: {flattened}") # [1, 2, 3, 4, 5, 6, 7]
```

Listing 10.2: Tree Traversal using yield from

10.5 Example: Generators for Large File Processing

```
1 import tracemalloc
2
3 def generate_sample_log(filename, num_lines=100000):
4     """Create a sample log file for testing"""
5     with open(filename, 'w') as f:
6         for i in range(num_lines):
7             if i % 10000 == 0:
```

```

8         f.write(f"ERROR: Critical error at line {i}\n")
9     elif i % 1000 == 0:
10         f.write(f"WARNING: Warning at line {i}\n")
11     else:
12         f.write(f"INFO: Normal operation at line {i}\n")
13
14 # Regular approach - loads entire file into memory
15 def read_all_errors_regular(filename):
16     """Reads entire file, filters errors - memory intensive"""
17     errors = []
18     with open(filename, 'r') as f:
19         for line in f:
20             if "ERROR" in line:
21                 errors.append(line.strip())
22     return errors
23
24 # Generator approach - yields errors one at a time
25 def read_all_errors_generator(filename):
26     """Generator that yields errors lazily - memory efficient"""
27     with open(filename, 'r') as f:
28         for line in f:
29             if "ERROR" in line:
30                 yield line.strip()
31
32 # Demonstrate memory comparison
33 generate_sample_log("sample_log.txt", 100000)
34
35 # Memory usage comparison
36 tracemalloc.start()
37 errors_regular = read_all_errors_regular("sample_log.txt")
38 current, peak = tracemalloc.get_traced_memory()
39 print(f"Regular approach peak memory: {peak / 1024:.2f} KB")
40 tracemalloc.stop()
41
42 tracemalloc.start()
43 errors_gen = read_all_errors_generator("sample_log.txt")
44 # Convert to list just for comparison (defeats purpose of generator)
45 errors_list = list(errors_gen)
46 current, peak = tracemalloc.get_traced_memory()
47 print(f"Generator approach peak memory: {peak / 1024:.2f} KB")
48 tracemalloc.stop()
49
50 import os
51 os.remove("sample_log.txt")

```

Listing 10.3: Memory-Efficient Log File Processing

10.6 Generator Methods

Method	Purpose	Example
next(gen)	Get next value	value = next(gen)
gen.send(value)	Send value to generator	gen.send(10)
gen.throw(exception)	Throw exception at generator	gen.throw(ValueError)
gen.close()	Stop generator	gen.close()
yield from	Delegate to subgenerator	yield from subgen()

Chapter 11

Error Handling in Python

Book Definitions

Source: "Python Programming: An Introduction to Computer Science" by John Zelle

"An error (or exception) is an event that occurs during the execution of a program that disrupts the normal flow of instructions. Python's exception handling mechanism allows programs to detect and recover from errors gracefully, using try-except blocks to catch and handle exceptions without crashing."

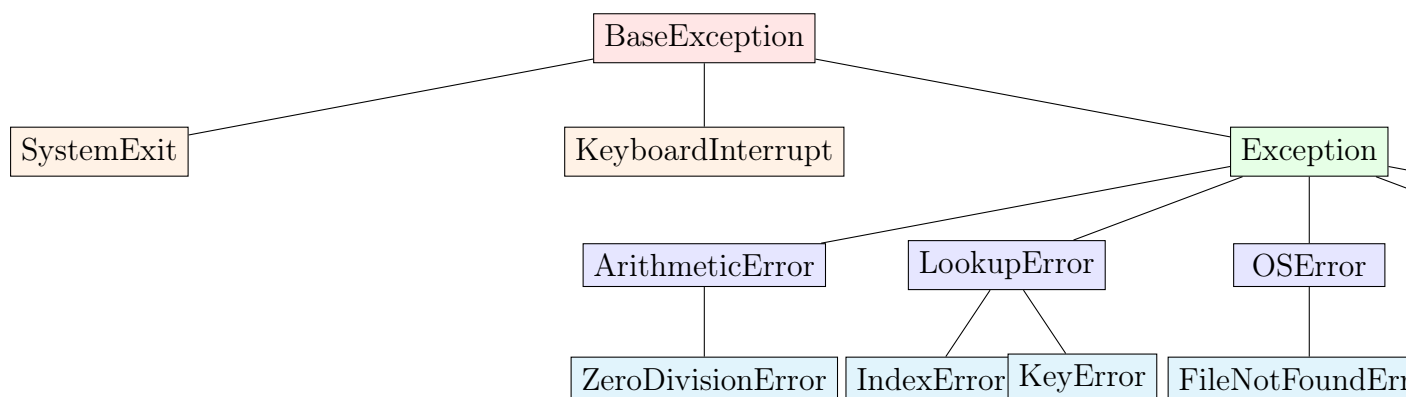
Source: Python Official Documentation

"Errors detected during execution are called exceptions and are not unconditionally fatal. Most exceptions are handled by the program, preventing them from terminating the program."

11.1 Simple Explanation

Errors are problems that occur when Python runs your code. Instead of letting the program crash, Python lets you "catch" these errors and handle them properly. This is like having a **safety net** - when something goes wrong, you can decide what to do next instead of the program just stopping.

11.2 Python Exception Hierarchy



11.3 Example: Complete Error Handling with Custom Exceptions

```
1 # Define custom exception hierarchy
2 class BankError(Exception):
3     """Base class for all bank-related exceptions"""
4     pass
5
6 class InsufficientFundsError(BankError):
7     """Raised when account balance is insufficient for withdrawal"""
8     def __init__(self, balance, amount):
9         self.balance = balance
10        self.amount = amount
11        super().__init__(
12            f"Insufficient funds! Balance: ${balance:,.2f}, "
13            f"Requested: ${amount:,.2f}. "
14            f"Shortfall: ${amount - balance:,.2f}"
15        )
16
17 class InvalidAmountError(BankError):
18     """Raised when transaction amount is invalid"""
19     pass
20
21 class AccountFrozenError(BankError):
22     """Raised when account is frozen"""
23     pass
24
25 class BankAccount:
26     def __init__(self, account_number, owner, balance=0):
27         self.account_number = account_number
28         self.owner = owner
29         self.balance = balance
30         self.frozen = False
31         self.transactions = []
32
33     def withdraw(self, amount):
34         """Withdraw money with proper error handling"""
35         try:
36             # Validate amount
37             if amount <= 0:
38                 raise InvalidAmountError("Withdrawal amount must be
39 positive")
40
41             if self.frozen:
42                 raise AccountFrozenError(f"Account {self.account_number
43 } is frozen")
44
45             if amount > self.balance:
46                 raise InsufficientFundsError(self.balance, amount)
47
48             # Process withdrawal
49             self.balance -= amount
50             self.transactions.append({
51                 'type': 'withdrawal',
52                 'amount': amount,
53                 'balance': self.balance
54             })
```

```

53         print(f"Withdrew ${amount:,.2f}. New balance: ${self.
balance:,.2f}")
54         return amount
55
56     except InvalidAmountError as e:
57         print(f"Invalid transaction: {e}")
58         return 0
59     except AccountFrozenError as e:
60         print(f"Account locked: {e}")
61         return 0
62     except InsufficientFundsError as e:
63         print(f"Funds error: {e}")
64         return 0
65
66     def deposit(self, amount):
67         """Deposit money with validation"""
68         if amount <= 0:
69             raise InvalidAmountError("Deposit amount must be positive")
70
71         if self.frozen:
72             raise AccountFrozenError(f"Cannot deposit to frozen account
{self.account_number}")
73
74         self.balance += amount
75         self.transactions.append({
76             'type': 'deposit',
77             'amount': amount,
78             'balance': self.balance
79         })
80         print(f"Deposited ${amount:,.2f}. New balance: ${self.balance
:,.2f}")
81         return amount
82
83     def freeze_account(self):
84         self.frozen = True
85         print(f"Account {self.account_number} has been frozen")
86
87     def unfreeze_account(self):
88         self.frozen = False
89         print(f"Account {self.account_number} has been unfrozen")
90
91     def get_transaction_history(self):
92         return self.transactions
93
94 # Testing the bank account
95 account = BankAccount("1234567890", "Alice Johnson", 1000)
96
97 print("=== Bank Account Operations ===")
98 account.deposit(500)                # Valid deposit
99 account.withdraw(200)               # Valid withdrawal
100 account.withdraw(2000)              # InsufficientFundsError
101 account.withdraw(-100)              # InvalidAmountError
102 account.freeze_account()            # Freeze account
103 account.withdraw(100)               # AccountFrozenError
104 account.unfreeze_account()          # Unfreeze
105 account.withdraw(100)               # Valid after unfreeze
106
107 print(f"\nFinal balance: ${account.balance:,.2f}")

```

```
108 print(f"Transaction count: {len(account.get_transaction_history())}")
```

Listing 11.1: Bank Account with Custom Exception Handling

11.4 The try-except-else-finally Structure

```
1 def read_file_with_handling(filename):
2     """Complete file reading with all exception handling blocks"""
3     file = None
4
5     try:
6         print(f"Attempting to open '{filename}'...")
7         file = open(filename, 'r')
8         content = file.read()
9
10    except FileNotFoundError:
11        print(f"Error: File '{filename}' not found")
12        return None
13
14    except PermissionError:
15        print(f"Error: Permission denied to read '{filename}'")
16        return None
17
18    except Exception as e:
19        print(f"Unexpected error: {type(e).__name__}: {e}")
20        return None
21
22    else:
23        # Executes ONLY if no exception occurred
24        print(f"Successfully read {len(content)} characters")
25        return content
26
27    finally:
28        # ALWAYS executes - used for cleanup
29        if file and not file.closed:
30            file.close()
31            print(f"File '{filename}' closed")
32
33    # Test with different scenarios
34    print("=== File Reading with Error Handling ===")
35    read_file_with_handling("C:/Windows/System32/config/SAM")
36    read_file_with_handling("/etc/shadow")
```

Listing 11.2: Demonstrating try-except-else-finally

11.5 Exception Handling Best Practices

```
1 # BAD PRACTICE 1: Catching all exceptions silently
2 def bad_example_1():
3     try:
4         result = 10 / 0
5         value = undefined_variable
6         int("abc")
7     except:
```

```

8         pass # Silently ignoring ALL errors!
9
10 # BAD PRACTICE 2: Catching too broadly
11 def bad_example_2():
12     try:
13         result = 10 / 0
14     except Exception as e:
15         print(f"Error: {e}")
16         # But was it ZeroDivisionError, TypeError, or something else?
17
18 # GOOD PRACTICE: Catch specific exceptions
19 def good_example_1():
20     try:
21         result = 10 / 0
22     except ZeroDivisionError:
23         print("Cannot divide by zero!")
24     except TypeError:
25         print("Type error in division!")
26     except Exception as e:
27         print(f"Unexpected error: {e}")
28
29 # GOOD PRACTICE: Always clean up resources
30 def good_example_2(filename):
31     file = None
32     try:
33         file = open(filename, 'r')
34         return file.read()
35     except FileNotFoundError:
36         print(f"File not found: {filename}")
37         return None
38     finally:
39         if file:
40             file.close()
41
42 # BEST PRACTICE: Use context managers (with statement)
43 def best_example(filename):
44     try:
45         with open(filename, 'r') as file:
46             return file.read()
47     except FileNotFoundError:
48         print(f"File not found: {filename}")
49         return None
50     # File automatically closed!

```

Listing 11.3: Best Practices for Exception Handling

Chapter 12

Context Managers in Python

Book Definitions

Source: Python Official Documentation

"A context manager is an object that defines the runtime context to be established when executing a with statement. The context manager handles the entry into, and the exit from, the desired runtime context for the execution of the block of code."

12.1 Simple Explanation

Context managers provide a way to properly manage resources like files, database connections, or locks. They ensure that resources are properly acquired and released, even if an error occurs in between. The most common example is opening files - the `with` statement automatically closes the file when you're done.

12.2 Example: Custom Context Manager

```
1 import time
2 from contextlib import contextmanager
3
4 # Class-based context manager
5 class Timer:
6     """Context manager to measure execution time"""
7
8     def __init__(self, description="Operation"):
9         self.description = description
10        self.start_time = None
11        self.end_time = None
12
13    def __enter__(self):
14        self.start_time = time.time()
15        print(f"Starting {self.description}...")
16        return self
17
18    def __exit__(self, exc_type, exc_val, exc_tb):
19        self.end_time = time.time()
20        elapsed = self.end_time - self.start_time
```

```

21         print(f"{self.description} completed in {elapsed:.4f} seconds")
22
23     # Return False to propagate exceptions, True to suppress
24     if exc_type is not None:
25         print(f"Error occurred: {exc_type.__name__}: {exc_val}")
26     return False # Don't suppress exceptions
27
28 # Function-based context manager using decorator
29 @contextmanager
30 def temporary_attribute(obj, name, value):
31     """Temporarily set an attribute on an object"""
32     original = getattr(obj, name, None)
33     setattr(obj, name, value)
34     try:
35         yield obj
36     finally:
37         if original is None:
38             delattr(obj, name)
39         else:
40             setattr(obj, name, original)
41
42 # Using the context managers
43 class Config:
44     def __init__(self):
45         self.debug = False
46         self.log_level = "INFO"
47
48 config = Config()
49 print(f"Debug before: {config.debug}") # False
50
51 # Using Timer context manager
52 with Timer("Complex calculation"):
53     total = sum(range(1000000))
54     print(f"Sum: {total}")
55
56 # Using temporary_attribute context manager
57 with temporary_attribute(config, 'debug', True):
58     print(f"Debug during: {config.debug}") # True
59
60 print(f"Debug after: {config.debug}") # False (restored)

```

Listing 12.1: Custom Context Manager Class and Decorator

Chapter 13

Decorators in Python

Book Definitions

Source: "Fluent Python" by Luciano Ramalho

"A decorator is a callable that takes another function as argument and returns a new function, usually replacing the original function with the new one. Decorators are a form of metaprogramming and are used to modify or enhance functions or methods without changing their source code."

13.1 Simple Explanation

A decorator is like a **wrapper** around a function that adds functionality before and/or after the function runs - without modifying the function itself. They let you "decorate" or "dress up" your functions with extra features.

13.2 Example: Practical Decorators

```
1 import time
2 import functools
3
4 # Decorator 1: Timer
5 def timer(func):
6     """Measure execution time of a function"""
7     @functools.wraps(func)
8     def wrapper(*args, **kwargs):
9         start = time.time()
10        result = func(*args, **kwargs)
11        end = time.time()
12        print(f"{func.__name__} took {end - start:.4f} seconds")
13        return result
14    return wrapper
15
16 # Decorator 2: Retry on failure
17 def retry(max_attempts=3, delay=0.1):
18     """Retry a function if it raises an exception"""
19     def decorator(func):
20         @functools.wraps(func)
21         def wrapper(*args, **kwargs):
```

```

22         attempts = 0
23         while attempts < max_attempts:
24             try:
25                 return func(*args, **kwargs)
26             except Exception as e:
27                 attempts += 1
28                 if attempts == max_attempts:
29                     raise
30                 print(f"Attempt {attempts} failed: {e}. Retrying...
31             ")
32             time.sleep(delay)
33         return None
34     return wrapper
35     return decorator
36
37 # Decorator 3: Cache results (memoization)
38 def memoize(func):
39     """Cache function results based on arguments"""
40     cache = {}
41     @functools.wraps(func)
42     def wrapper(*args):
43         if args not in cache:
44             cache[args] = func(*args)
45         return cache[args]
46     return wrapper
47
48 # Decorator 4: Input validation
49 def validate_positive(func):
50     """Ensure all numeric arguments are positive"""
51     @functools.wraps(func)
52     def wrapper(*args, **kwargs):
53         for arg in args:
54             if isinstance(arg, (int, float)) and arg < 0:
55                 raise ValueError(f"Negative argument not allowed: {arg}")
56         return func(*args, **kwargs)
57     return wrapper
58
59 # Decorator 5: Class-based decorator
60 class CountCalls:
61     """Decorator that counts how many times a function is called"""
62     def __init__(self, func):
63         functools.update_wrapper(self, func)
64         self.func = func
65         self.count = 0
66
67     def __call__(self, *args, **kwargs):
68         self.count += 1
69         print(f"Call {self.count} to {self.func.__name__}")
70         return self.func(*args, **kwargs)
71
72 # Using the decorators
73 @timer
74 @memoize
75 @validate_positive
76 def fibonacci(n):
77     """Calculate nth Fibonacci number"""

```

```

78     if n <= 1:
79         return n
80     return fibonacci(n-1) + fibonacci(n-2)
81
82 @retry(max_attempts=3, delay=0.1)
83 def unstable_network_call():
84     """Simulate an unstable network"""
85     import random
86     if random.random() < 0.7:
87         raise ConnectionError("Network timeout")
88     return "Success!"
89
90 @CountCalls
91 def greet(name):
92     """Greet someone"""
93     return f"Hello, {name}!"
94
95 # Test all decorators
96 print("=== Testing Decorators ===")
97
98 # Test timer + memoize + validate_positive
99 print("\n1. Fibonacci with timer, memoize, and validation:")
100 print(f"fibonacci(10) = {fibonacci(10)}")
101 print(f"fibonacci(10) = {fibonacci(10)}") # Cached result
102
103 # Test retry
104 print("\n2. Retry decorator:")
105 try:
106     result = unstable_network_call()
107     print(f"Result: {result}")
108 except ConnectionError:
109     print("Failed after all retries")
110
111 # Test call counter
112 print("\n3. Call counter decorator:")
113 print(greet("Alice"))
114 print(greet("Bob"))
115 print(greet("Charlie"))
116 print(f"greet() was called {greet.count} times")

```

Listing 13.1: Practical Decorators for Logging, Timing, and Caching

Chapter 14

Descriptors and Properties

Book Definitions

Source: Python Official Documentation

"In general, a descriptor is an object attribute with 'binding behavior', one whose attribute access has been overridden by methods in the descriptor protocol: `__get__()`, `__set__()`, and `__delete__()`. Properties are a simpler way to create descriptors using the `@property` decorator."

14.1 Example: Property Decorator and Custom Descriptor

```
1 # Using @property decorator (simple descriptor)
2 class Temperature:
3     def __init__(self, celsius=0):
4         self._celsius = celsius
5
6     @property
7     def celsius(self):
8         """Get temperature in Celsius"""
9         return self._celsius
10
11     @celsius.setter
12     def celsius(self, value):
13         """Set temperature in Celsius"""
14         if value < -273.15:
15             raise ValueError("Temperature below absolute zero!")
16         self._celsius = value
17
18     @property
19     def fahrenheit(self):
20         """Convert Celsius to Fahrenheit"""
21         return (self._celsius * 9/5) + 32
22
23     @fahrenheit.setter
24     def fahrenheit(self, value):
25         """Set temperature using Fahrenheit"""
26         self.celsius = (value - 32) * 5/9
27
```

```

28     @property
29     def kelvin(self):
30         """Convert Celsius to Kelvin (read-only)"""
31         return self._celsius + 273.15
32
33 # Custom descriptor class
34 class ValidatedString:
35     """Descriptor that validates string length"""
36
37     def __init__(self, min_length=1, max_length=100):
38         self.min_length = min_length
39         self.max_length = max_length
40         self.data = {}
41
42     def __get__(self, instance, owner):
43         if instance is None:
44             return self
45         return self.data.get(instance, '')
46
47     def __set__(self, instance, value):
48         if not isinstance(value, str):
49             raise TypeError(f"Expected str, got {type(value).__name__}")
50
51         if len(value) < self.min_length:
52             raise ValueError(f"String must be at least {self.min_length} chars")
53         if len(value) > self.max_length:
54             raise ValueError(f"String must be at most {self.max_length} chars")
55         self.data[instance] = value
56
57     def __delete__(self, instance):
58         del self.data[instance]
59
60 class Person:
61     # Using custom descriptor
62     name = ValidatedString(min_length=2, max_length=50)
63     email = ValidatedString(min_length=5, max_length=100)
64
65     def __init__(self, name, email, temperature=None):
66         self.name = name
67         self.email = email
68         self.temperature = Temperature(temperature if temperature else
25)
69
70     def __str__(self):
71         return (f"Person(name='{self.name}', email='{self.email}', "
72                 f"temp={self.temperature.celsius} C)")
73
74 # Testing
75 print("=== Properties Demo ===")
76 temp = Temperature(25)
77 print(f"Celsius: {temp.celsius} C ")          # 25 C
78 print(f"Fahrenheit: {temp.fahrenheit} F ")    # 77 F
79 print(f"Kelvin: {temp.kelvin}K")              # 298.15K
80
81 temp.fahrenheit = 32 # Set using Fahrenheit
82 print(f"After setting to 32 F :")

```

```

82 print(f"Celsius: {temp.celsius} C ")          # 0 C
83
84 print("\n=== Descriptors Demo ===")
85 person = Person("Alice", "alice@example.com", 20)
86 print(person)
87
88 try:
89     person.name = "A" # Too short
90 except ValueError as e:
91     print(f"Validation error: {e}")
92
93 try:
94     person.name = 123 # Not a string
95 except TypeError as e:
96     print(f"Type error: {e}")

```

Listing 14.1: Properties and Descriptors in Practice

Chapter 15

Metaclasses in Python

Book Definitions

Source: "Fluent Python" by Luciano Ramalho

"A metaclass is a class whose instances are classes. Just as a regular class defines the behavior of its instances, a metaclass defines the behavior of classes and their instances. Metaclasses allow you to customize class creation, modify class attributes, and enforce constraints on classes."

15.1 Simple Explanation

If classes are **blueprints** for objects, metaclasses are **blueprints for classes**. They control how classes are created - like a factory that produces classes instead of objects.

- `type` is the default metaclass in Python
- Metaclasses let you intercept and modify class creation
- Used for frameworks, ORMs, and enforcing coding standards

15.2 Example: Using Metaclasses

```
1 # Metaclass 1: Auto-register subclasses
2 class AutoRegisterMeta(type):
3     """Metaclass that automatically registers all subclasses"""
4     registry = {}
5
6     def __new__(mcs, name, bases, namespace):
7         cls = super().__new__(mcs, name, bases, namespace)
8         if name != 'Base': # Don't register the base class
9             mcs.registry[name.lower()] = cls
10        return cls
11
12    @classmethod
13    def get_registry(mcs):
14        return mcs.registry
15
16 # Metaclass 2: Singleton pattern
```

```

17 class SingletonMeta(type):
18     """Metaclass that implements the Singleton pattern"""
19     _instances = {}
20
21     def __call__(cls, *args, **kwargs):
22         if cls not in cls._instances:
23             cls._instances[cls] = super().__call__(*args, **kwargs)
24         return cls._instances[cls]
25
26 # Using AutoRegisterMeta
27 class Animal(metaclass=AutoRegisterMeta):
28     """Base class for animals - abstract"""
29     def speak(self):
30         raise NotImplementedError
31
32 class Dog(Animal):
33     def speak(self):
34         return "Woof!"
35
36 class Cat(Animal):
37     def speak(self):
38         return "Meow!"
39
40 class Bird(Animal):
41     def speak(self):
42         return "Chirp!"
43
44 # Check registry
45 print("Registered animals:", list(Animal.get_registry().keys()))
46
47 # Create animal by type
48 def create_animal(animal_type):
49     """Factory function using auto-registration"""
50     animal_class = Animal.get_registry().get(animal_type.lower())
51     if animal_class:
52         return animal_class()
53     raise ValueError(f"Unknown animal: {animal_type}")
54
55 dog = create_animal("dog")
56 cat = create_animal("cat")
57 print(f"Dog says: {dog.speak()}")
58 print(f"Cat says: {cat.speak()}")
59
60 # Using SingletonMeta
61 class DatabaseConnection(metaclass=SingletonMeta):
62     def __init__(self, connection_string=None):
63         self.connection_string = connection_string
64         self.connected = False
65
66     def connect(self):
67         if not self.connected:
68             self.connected = True
69             print(f"Connected to: {self.connection_string}")
70         else:
71             print("Already connected")
72
73     def disconnect(self):
74         self.connected = False

```

```

75         print("Disconnected")
76
77 # Test singleton behavior
78 db1 = DatabaseConnection("mysql://localhost:3306")
79 db2 = DatabaseConnection("postgresql://localhost:5432")
80
81 print(f"\ndb1 is db2: {db1 is db2}") # True - same instance!
82 print(f"db1 connection: {db1.connection_string}") # mysql://...
83 print(f"db2 connection: {db2.connection_string}") # mysql://... (same
    !)
```

Listing 15.1: Metaclass for Auto-Registration and Validation

Chapter 16

Asynchronous Programming with asyncio

Book Definitions

Source: Python Official Documentation

"asyncio is a library to write concurrent code using the async/await syntax. It is used as a foundation for multiple Python asynchronous frameworks that provide high-performance network and web-servers, database connection libraries, distributed task queues, etc."

16.1 Example: Async/Await with asyncio

```
1 import asyncio
2 import time
3
4 # Synchronous version (blocking)
5 def fetch_data_sync(id, delay):
6     """Simulate synchronous data fetching"""
7     print(f"Starting sync fetch {id}...")
8     time.sleep(delay) # Blocks the entire thread!
9     print(f"Finished sync fetch {id}")
10    return f"Data-{id}"
11
12 # Asynchronous version (non-blocking)
13 async def fetch_data_async(id, delay):
14     """Simulate asynchronous data fetching"""
15     print(f"Starting async fetch {id}...")
16     await asyncio.sleep(delay) # Non-blocking sleep
17     print(f"Finished async fetch {id}")
18     return f"Data-{id}"
19
20 # Async context manager
21 class AsyncResource:
22     """Demonstrates async context manager"""
23
24     async def __aenter__(self):
25         print("Acquiring async resource...")
26         await asyncio.sleep(0.1)
27         return self
```

```

28
29     async def __aexit__(self, exc_type, exc_val, exc_tb):
30         print("Releasing async resource...")
31         await asyncio.sleep(0.1)
32
33     async def process(self, data):
34         await asyncio.sleep(0.5)
35         return data.upper()
36
37 # Async generator
38 async def async_generator(limit):
39     """Asynchronous generator that yields values with delay"""
40     for i in range(limit):
41         await asyncio.sleep(0.2)
42         yield i
43
44 async def main():
45     """Main async function demonstrating various features"""
46
47     # Sequential execution
48     print("\n=== Sequential Async Calls ===")
49     start = time.time()
50     result1 = await fetch_data_async(1, 1)
51     result2 = await fetch_data_async(2, 1)
52     result3 = await fetch_data_async(3, 1)
53     print(f"Sequential took: {time.time() - start:.2f}s")
54     print(f"Results: {result1}, {result2}, {result3}")
55
56     # Concurrent execution with gather
57     print("\n=== Concurrent Async Calls ===")
58     start = time.time()
59     results = await asyncio.gather(
60         fetch_data_async(1, 1),
61         fetch_data_async(2, 1),
62         fetch_data_async(3, 1)
63     )
64     print(f"Concurrent took: {time.time() - start:.2f}s")
65     print(f"Results: {results}")
66
67     # Using async context manager
68     print("\n=== Async Context Manager ===")
69     async with AsyncResource() as resource:
70         result = await resource.process("hello async")
71         print(f"Processed: {result}")
72
73     # Using async generator
74     print("\n=== Async Generator ===")
75     async for value in async_generator(5):
76         print(f"Generated: {value}")
77
78     # Task-based concurrency
79     print("\n=== Task-Based Concurrency ===")
80     task1 = asyncio.create_task(fetch_data_async("A", 1))
81     task2 = asyncio.create_task(fetch_data_async("B", 0.5))
82     task3 = asyncio.create_task(fetch_data_async("C", 0.8))
83
84     # Wait for all tasks to complete
85     task_results = await asyncio.gather(task1, task2, task3)

```

```

86     print(f"Task results: {task_results}")
87
88     # Wait for first task to complete
89     print("\n=== Wait for First Task ===")
90     t1 = asyncio.create_task(fetch_data_async("X", 2))
91     t2 = asyncio.create_task(fetch_data_async("Y", 0.5))
92
93     done, pending = await asyncio.wait(
94         {t1, t2},
95         return_when=asyncio.FIRST_COMPLETED
96     )
97
98     for task in done:
99         print(f"First completed: {task.result()}")
100
101     # Cancel pending tasks
102     for task in pending:
103         task.cancel()
104
105 # Run the async main function
106 print("=== Asyncio Demo ===")
107 asyncio.run(main())

```

Listing 16.1: Asynchronous Programming with asyncio

Chapter 17

Python Advanced Concepts Cheat Sheet

17.1 Operator Overloading Quick Reference

Operator	Method	Description
+	<code>--add--</code>	Addition
-	<code>--sub--</code>	Subtraction
*	<code>--mul--</code>	Multiplication
/	<code>--truediv--</code>	Division
<code>==</code>	<code>--eq--</code>	Equality
<code><</code>	<code>--lt--</code>	Less than
<code>></code>	<code>--gt--</code>	Greater than
<code>len()</code>	<code>--len--</code>	Length
<code>str()</code>	<code>--str--</code>	String

17.2 Iterator vs Generator vs Generator Expression

Feature	Iterator	Generator
Creation	Class with <code>--next--</code>	Function with <code>yield</code>
State	Manual state management	Automatic state preservation
Code Size	More verbose	Concise
Memory	Low	Very low
Speed	Slightly faster	Slightly slower

Keyword	Description
<code>try</code>	Block where exceptions may occur
<code>except</code>	Catch and handle specific exceptions
<code>else</code>	Executes if no exception in try block
<code>finally</code>	Always executes (cleanup)
<code>raise</code>	Explicitly raise an exception
<code>assert</code>	Raise <code>AssertionError</code> if condition is False

Method	Purpose
<code>__init__(self, ...)</code>	Constructor
<code>__str__(self)</code>	User-friendly string representation
<code>__repr__(self)</code>	Developer string representation
<code>__len__(self)</code>	Length (for <code>len()</code>)
<code>__getitem__(self, key)</code>	Indexing (<code>obj[key]</code>)
<code>__setitem__(self, key, value)</code>	Setting item (<code>obj[key] = value</code>)
<code>__iter__(self)</code>	Make object iterable
<code>__next__(self)</code>	Get next item (for iterators)
<code>__enter__(self)</code>	Context manager entry
<code>__exit__(self, ...)</code>	Context manager exit
<code>__call__(self, ...)</code>	Make object callable
<code>__hash__(self)</code>	Hash value (for dict keys)
<code>__eq__(self, other)</code>	Equality comparison

17.3 Exception Handling Keywords

17.4 Common Python Magic Methods

17.5 Important Built-in Decorators

- `@staticmethod` - Method that doesn't access instance or class
- `@classmethod` - Method that receives class as first argument
- `@property` - Turn method into attribute-like access
- `@abstractmethod` - Mark method as abstract (requires implementation)
- `@functools.wraps` - Preserve original function metadata
- `@functools.lru_cache` - Cache function results
- `@contextmanager` - Create context manager from generator
- `@dataclass` - Auto-generate `__init__`, `__repr__`, etc.

17.6 ABC (Abstract Base Classes) Quick Reference

```

1 from abc import ABC, abstractmethod
2
3 class MyABC(ABC):
4     @abstractmethod
5     def required_method(self):
6         """Must be implemented by subclasses"""
7         pass
8
9     def concrete_method(self):
10        """Can be used as-is or overridden"""
11        return "Default implementation"
12
13 # Subclass must implement required_method
14 class MyClass(MyABC):
15     def required_method(self):
16         return "Implemented!"

```

Listing 17.1: ABC Pattern